When Does GRPO-Style Training Have a Learning Signal? A Claim-to-Evidence Audit of a Critic-Free Group-Relative Runner

Anonymous Author(s)
Affiliation
Address
email

Abstract

This paper is a claim-to-evidence audit, not a benchmark or leaderboard. We study whether critic-free group-relative reinforcement learning for LLMs has a learning signal under a given model–reward–sampler–stack combination, and how not to overclaim training reward as capability when it does not.

First, we introduce a mixed-group probability mechanism:

$$P(\text{usable}) \approx \frac{1}{N} \sum_{x} \left[1 - (1 - p_x)^G - p_x^G\right],$$

where p_x is the current policy's success rate on prompt x and G is the group size. This shows that a group-relative update has useful signal only when the policy samples mixed-reward groups; all-identical rewards yield zero gradient. The familiar $p_x > 1/G$ rule is only an expected-one-success heuristic, not a threshold theorem.

Second, we operationalize this as Zero-Variance Fraction (ZVF) and Gradient Utilization (GU) as *triage diagnostics*, not capability metrics. ZVF measures the fraction of prompts where all G completions receive identical rewards; GU $= 1-$ZVF measures usable signal fraction. High ZVF with low reward indicates cold-start collapse; high ZVF with high reward indicates saturation; low ZVF indicates within-group contrast. A fixed first-five-step rule (ZVF $\geq 80\%$ with reward $\leq 5\%$) catches all collapsed runs in a 22-run validation set (precision 1.0, recall 1.0), but only two positive failures are observed and continuous ZVF ranking is weak. The defensible claim is triage, not prediction. Importantly, ZVF is task-driven: tool-use experiments saturate at ZVF $= 100\%$ regardless of model scale, while GSM8K experiments average ZVF $= 8.5\%$.

Third, we show that training reward conflates with held-out capability in most prior work and in this corpus. The only clean held-out capability result is a paired GSM8K 200-example evaluation: Qwen3-8B improves only $82.0\% \rightarrow 83.3\%$ ($p = 0.26$), a small and non-significant gain. Checkpoint-selected held-out evaluations (top-10 by training last-10) yield a 87.2–95.0% band, but selection by training reward introduces over-optimism. Four Llama-3.3-70B seeds show training last-10 values ranging 95.0–98.1% yet all land on 95.0% held-out accuracy, demonstrating that last-10 variance reflects Monte-Carlo noise rather than genuine generalization differences.

Fourth, we show that algorithm labels (PPO, GRPO, DPO) are under-specified experimental treatments. Under nominally matched visible configurations (Qwen3-8B, $G = 8$, lr$= 10^{-5}$, GSM8K, 30 steps, seed 42), the Tinker runner reaches

Submitted to 39th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Table 1: Claim–evidence–verdict hierarchy.

Claim	Status	Headline evidence	Verdict
C1: The runner is a reward-contrast amplifier.	Mechanistic + operational	Mixed-group probability formula; ZVF/GU triage (collapse/saturation/contrast).	Signal exists only when sampled groups contain reward diversity.
C2: ZVF/GU provide early triage diagnostics.	Most operational contribution	First-5-step rule (ZVF $\geq 80\%$, reward $\leq 5\%$) catches all collapsed runs (precision 1.0, recall 1.0, 22-run validation). ZVF is task-driven.	Triage, not prediction. Not a calibrated performance model.
C3: Training reward conflates with held-out capability.	Central negative result	Held-out GSM8K $82.0\% \rightarrow 83.3\%$, $p = 0.26$; checkpoint-selection overoptimism.	Training reward supports dynamics claims only.
C4: Algorithm labels (PPO, GRPO, DPO) are underspecified.	Methodological warning	$17\times$ last-10 gap under nominally matched configs; artifact sensitivity.	Full-stack reporting required.

84.4% last-10 training reward while TRL on Modal H100 reaches 5.0%. This $17\times$ gap does not rank frameworks; it proves that backend, sampler, reward parser, LoRA configuration, optimizer, and evaluator are co-determinants of outcome in ways that hyperparameter tables do not capture.

We do not claim this runner broadly improves reasoning. We do not claim ZVF predicts final performance. We do not claim a universal PPO-versus-GRPO ranking. Our contribution is a diagnostic protocol: how to decide whether a specific model–reward–sampler–stack combination has usable group-relative RL signal, and how to scope claims accordingly. We release the code, run logs, and checkpoints at `https://anonymous.4open.science/r/tinker-rl-lab`.

1 Introduction

PPO [108], GRPO [110], and DPO [103] are often compared as if the algorithm label were the full experimental treatment. In LLM post-training, outcomes also depend on base model, instruction tuning, reward parser, group construction, sampler, backend, LoRA configuration, checkpoint selection, and evaluation harness. This is the LLM post-training analogue of the reproducibility concerns Henderson et al. identified for deep RL [32]: algorithm labels are under-specified unless the full training stack is fixed or modeled.

This paper audits a group-relative RL runner inspired by GRPO, not a canonical GRPO reproduction. It retains group-normalized advantages and a critic-free structure, but does not implement the PPO ratio and clip of Schulman et al. [108], does not retain a frozen reference policy for KL regularization, performs one optimizer step per rollout batch, and does not use a completion-only token mask. A P1-A diagnostic on Qwen3.5-4B measures 61.6–89.6% of the full-sequence loss magnitude from prompt tokens rather than completion tokens. Claims in this paper concern this runner, not canonical GRPO.

The paper does not show that this runner broadly improves reasoning. The only clean held-out capability result is a paired GSM8K 200-example evaluation that improves only $82.0\% \rightarrow 83.3\%$ ($p = 0.26$); we do not claim a significant held-out math effect. The paper's positive contribution is a diagnostic protocol: how to decide whether a specific model–reward–sampler– stack combination has usable group-relative RL signal, and how to scope claims accordingly.

1.1 Claim 1: Reward-contrast amplifier mechanism.

A group-relative update has useful signal only when the sampled group contains distinguishable reward outcomes. Under binary or verifier-like rewards, if the current policy succeeds on prompt x

with probability p_x and group size is G, the probability of a mixed, gradient-carrying group is

$$P(\text{usable}) \approx \frac{1}{N} \sum_x \left[1 - (1 - p_x)^G - p_x^G\right].$$

The familiar $p_x > 1/G$ rule is only an expected-one-success heuristic; it is not a threshold theorem. This mechanism explains why warm-starts, group size, reward density, and task difficulty matter more than task labels. When ZVF $= 1$ (all groups identical-reward), no gradient flows; when ZVF $= 0$ (all groups mixed-reward), the update exploits within-group contrast.

1.2 Claim 2: ZVF/GU as triage diagnostics.

Zero-Variance Fraction (ZVF) measures the fraction of prompts where all G completions receive identical rewards, producing zero gradient. Gradient Utilization (GU) $= 1-$ZVF measures the fraction of prompts providing usable signal. High ZVF with low reward indicates cold-start collapse; high ZVF with high reward indicates saturation; low ZVF indicates within-group contrast. A fixed first-five-step rule (ZVF $\geq 80\%$, reward $\leq 5\%$) catches both collapsed runs in a 22-run de-duplicated validation set with no false positives, but only two positive failures are observed and continuous ZVF ranking is weak. The defensible claim is triage, not prediction: ZVF/GU can support early decisions to stop, warm-start, densify reward, adjust task difficulty, or change group size. Importantly, ZVF is task-driven, not model-scale-driven: tool-use experiments saturate at ZVF $= 100\%$ regardless of model size, while GSM8K experiments average ZVF $= 8.5\%$.

The mixed-group probability formula is the explanatory core; ZVF/GU is the operational interface.

1.3 Claim 3: Training reward is not capability.

Online reward curves measure the reward environment that produced the update. They should not be upgraded into general reasoning or tool-use claims without matched held-out evaluation. The clean GSM8K control improves only $82.0\% \rightarrow 83.3\%$ ($p = 0.26$), so we do not claim a significant held-out math effect. Held-out evaluation of the top-10 checkpoints (selected by training last-10 reward) yields an 87.2–95.0% accuracy band, but this is a checkpoint-selection analysis: selecting by training reward introduces over-optimism. Four Llama-3.3-70B-Instruct seeds range 95.0–98.1% training last-10 but all land on 95.0% held-out accuracy, showing that last-10 variance reflects Monte-Carlo sampling noise rather than genuine generalization differences.

Tool-use, HumanEval, and MATH rows remain lower-tier evidence: tool-use scores schema compliance (not executed success), the HumanEval sandbox removes Python built-ins, and some prompt pools come from test splits used as reward-environment probes. These are useful as dynamics evidence and as sparse- reward failure cases, not as proof of agentic capability.

1.4 Claim 4: Algorithm labels are under-specified treatments.

Algorithm-level claims require full-stack specification. Under nominally matched visible configurations (Qwen3-8B, $G = 8$, lr$= 10^{-5}$, GSM8K, 30 steps, seed 42), the Tinker runner reaches 84.4% last-10 reward while TRL on Modal H100 reaches 5.0%. This $17\times$ gap does not prove Tinker is better than TRL; it proves that "same GRPO config" is under-specified unless backend, sampler, reward parser, LoRA configuration, optimizer, checkpoint-selection rule, and evaluator are reported and controlled. Our Qwen PPO/GRPO comparison is artifact-sensitive (PPO last-10 differs between ledger and statistical summary), and the Llama comparison favors PPO but is single-seed and backend-confounded. No universal PPO-versus-GRPO ranking is supported.

1.5 Contributions.

This paper contributes: (i) a mixed-group probability mechanism explaining when a group-relative RL loop has learning signal; (ii) ZVF/GU as early triage diagnostics separating cold-start collapse, saturation, and usable contrast; (iii) a held-out GSM8K result showing that training reward conflates with capability ($82.0\% \rightarrow 83.3\%$, $p = 0.26$); (iv) a methodological warning that PPO, GRPO, and DPO comparisons require full-stack specification; (v) a diagnostic protocol for deciding whether a specific model–reward– sampler–stack combination has usable group-relative RL signal. We release the code, run logs, and checkpoints at `https://anonymous.4open.science/r/tinker-rl-lab`.

2 Related Work

TINKERRL AUDIT is not an optimizer paper in the style of DeepSeekMath, nor a new zero-variance optimizer or prompt-selection rule in the style of RL-ZVP or Online Difficulty Filtering. It is a systems-oriented empirical audit of when group-relative LLM post-training has a usable reward-contrast signal and when its rewards become misleading evidence. We therefore position the work along five axes: policy-gradient *algorithms* for language models, RL for *reasoning* (including process reward models and spurious-reward studies), RL for *tool use* and agentic behaviour, descriptive scale and reward-trajectory studies for RL post-training, and the *frameworks*, *evaluation harnesses*, and reproducibility practices that instantiate these algorithms. The recurring claim is that an algorithm label is not the full treatment: the model family, reward parser, sampler, backend, mask, checkpoint rule, and evaluation split all condition what a reported RL gain means.

2.1 RL Algorithms for Large Language Models

Proximal Policy Optimization (PPO) [108] is the workhorse algorithm behind InstructGPT-style RLHF [91, 17, 118, 8, 9] and has dominated production deployments for five years. Direct Preference Optimization [103] reframed preference learning as contrastive classification and spawned a large family of RL-free alternatives, including IPO [6], KTO [25], SimPO [79], ORPO [39], and SLiC [154], all of which we consider as no-rollout baselines in our cross-library protocol.

The current wave is defined by *group-relative* policy gradient methods. DeepSeekMath [110] introduced GRPO together with a 7B math model trained from a math-heavy data pipeline; it framed GRPO as a memory-efficient PPO variant that samples multiple completions per prompt, uses group-relative rewards instead of a learned critic, and retains PPO-style ratio/clip and reference-policy regularization. DeepSeek-R1 later popularised outcome-reward reasoning RL at much larger scale [23, 24]. This paper uses DeepSeekMath as the canonical anchor for what a faithful "GRPO" claim would require, but our runner is deliberately narrower: it preserves the critic-free, group-relative reward-contrast mechanism while omitting several canonical implementation choices, including probability-ratio clipping, a frozen-reference KL term, and completion-only masking. Our claims are therefore mechanism-level and diagnostic rather than leaderboard-level: we study when a group-relative loop has reward diversity to amplify, not whether a particular DeepSeekMath recipe has been reproduced. A second wave of refinements dissects the biases of vanilla GRPO. Dr. GRPO [69] removes a response-length bias and a question-difficulty amplification bias by using a constant loss normaliser and dropping the standard-deviation denominator in the advantage. GDPO [68] decouples group-relative normalisation per reward component, preventing advantage collapse when rewards are multi-valued. MAD GRPO [132] replaces the standard-deviation advantage scaler with the Median Absolute Deviation for heavy-tailed reward distributions. G^{2}RPO [43] forces per-prompt advantages to match $\mathcal{N}(0,1)$ to fix reward-topology variance in multimodal settings. Wu et al. [135] prove that GRPO is "secretly DPO" — the group-level advantage is an implicit contrastive objective — and show that 2-GRPO retains 98.1% of the performance of 16-GRPO at 12.5% of the rollout budget. iGRPO [30] adds a two-stage self-feedback loop that conditions subsequent rollouts on the policy's best draft, reaching 85.6% on AIME24. AERO [149] combines adaptive rollout sizing, rejection sampling, and a Bayesian posterior over prompt difficulty to eliminate zero-advantage dead zones, cutting RL compute by 48%. Target Policy Optimization (TPO) [48] decouples target-distribution construction from policy fitting: given scored completions, TPO builds a target $q \propto p_{\text{old}} \exp(u)$ and fits the policy via cross-entropy, giving a zero-gradient fixed point once the policy matches the target. RGRA [12] shows that PPO-style clipping is unnecessary on top of group relative advantages, outperforming GRPO on 17 of 27 comparisons. EFRame [127] augments GRPO with exploration rollouts and experience replay for rare trajectories.

A parallel thread focuses on *production-scale* refinements. GSPO, from the Qwen team [155], redefines the importance ratio at the sequence level (rather than token level) and was credited with stabilising the Qwen3 series, especially its Mixture-of-Experts variants. DAPO, from ByteDance [144], contributes asymmetric Clip-Higher, dynamic sampling, token-level loss normalisation, and overlong filtering, reaching 50 points on AIME 2024 with a 32B model. Kimi-K1.5 [51, 50] and DeepSeek-V3 [22] both report their own GRPO variants and online mirror-descent-style updates. Orthogonal production variants include Flow-GRPO [64], VAPO [11], and Dr. SAC [3]; the last argues that classical REINFORCE with a value baseline, when carefully implemented, matches GRPO on several RLHF tasks.

The closest algorithmic relatives are works that respond directly to *zero-variance* or zero-advantage groups. RL-ZVP [54] argues that all-correct and all-incorrect groups need not be discarded: it reshapes the advantage with token-level signals so zero-variance prompts can still update the policy. Online Difficulty Filtering [7] takes the complementary route of changing the prompt stream, selecting tasks where the current policy's success probability is intermediate so sampled batches maintain high reward variance. These papers optimize around the same bottleneck that we measure. The distinction is that TINKERRL AUDIT does not propose a replacement optimizer or curriculum; it turns ZVF and Gradient Utilization (GU) into run-level diagnostics for ordinary group-relative experiments. High-ZVF/low-reward runs indicate cold-start collapse; high-ZVF/high-reward runs indicate saturation; low ZVF indicates usable within-group contrast. Thus RL-ZVP and Online Difficulty Filtering ask how to recover or select signal, while we ask whether a given stack currently has signal and what evidential status its reward curve deserves. Other proposals alter sampling or advantage estimation—AERO [149] and GRESO [148] pre-screen prompts, CPPO [62] prunes completions post-rollout, NGRPO [82] calibrates all-incorrect groups, Scaf-GRPO [150] injects hints, and MO-GRPO [45] re-weights multi-objective rewards. Theory work such as Demystifying GRPO [157], MinPRO [57], and SSPO [139] explains the estimator or granularity of the update; our contribution is the complementary experimental accounting layer that decides when those updates are being driven by real reward contrast.

2.2 Reasoning RL and Process Reward Models

The shift from outcome rewards to *process* supervision has reshaped reasoning RL. Let's-Verify-Step-by-Step [61] showed that step-level PRMs dramatically outperform outcome reward models on GSM8K and MATH. Math-Shepherd [129] derives process rewards from auto-generated process annotations, avoiding costly human labels. PRM800K [123] and PRMBench [117] provide evaluation data for step-level scoring. OmegaPRM [74] uses Monte-Carlo tree search to supervise process rewards without human-in-the-loop, while ReasonEval [136] introduces reasoning-specific reward models. Implicit PRM [145] learns PRMs implicitly from outcome labels, and VersaPRM [146] extends process rewards to multi-domain tasks. These supervision signals are the backbone of recent reasoning models, including OpenAI's o1/o3 family [87, 89], DeepSeek-R1 [23, 24], Qwen2.5-Math [138], Qwen3 [101], Gemini 2.5 Pro's deliberative mode [28], and Kimi-K2 [50]. Complementary results demonstrate that self-consistency [131], tree-of-thought search [142], and critic-style post-hoc verifiers [78] all benefit from or substitute for process reward supervision. VerifyBench [153] and A Sober Look [36] warn, however, that reward hacking and evaluation artefacts can inflate perceived gains. Spurious Rewards [109] is the closest negative control for our interpretation: it shows that RLVR can yield large math-benchmark gains on some model families even when the reward is random, format-only, incorrect, or otherwise weakly aligned with correctness, and that these effects do not necessarily transfer across model families. Our proxy-reward results are read under the same constraint. Online reward curves are evidence about training dynamics and reward-environment interaction; capability claims require held-out or shifted evaluations. This evidence hierarchy is therefore not a presentation choice but the paper's response to the spurious-reward literature.

Reasoning-specific algorithmic contributions also continue to accumulate. ReFT [75] studies supervised fine-tuning versus RL for reasoning. Let's Reward Step [152] and STaR [147] bootstrap reasoning chains from weak supervisors. rStar-Math [29] combines MCTS-based PRMs with self-evolved reasoning. ReST-EM [115] alternates expectation and maximisation steps over self-generated rationales. Self-Taught Evaluator [130] adapts an LLM to score its own reasoning chains. GRPO-Lead [72], LCPO [73], and LIMR [59] each tune reasoning depth with length-aware rewards. Search-R1 [46] trains reasoning agents to interleave search queries with chain-of-thought. TTRL [98] performs test-time RL against self-generated verifiers. We discuss the Dr. GRPO [69] and GSPO [155] formulations as related reference points for group-relative RL. Our own measurements across 0.6B–235B are primarily reward-harness diagnostics; held-out capability claims are limited to the explicitly separated GSM8K evaluation.

2.3 Tool-Use and Agentic RL

Tool-use RL trains LLMs to interleave natural-language reasoning with external API calls. Toolformer [107] introduced self-supervised tool-call insertion for a small set of APIs; Gorilla [94] retrieved from a large API catalogue; and ToolLLM [100] scaled training to thousands of real-world REST APIs. ReAct [141] established the interleaved reason–act–observe pattern that most subse-

quent agents inherit. RL specifically for tool use was pioneered by ToolACE [66], which couples a tool-calling reward with a self-evolving synthesis pipeline; ToolRL [99] applies GRPO with verifiable tool-call rewards; ToRL [60] optimises the joint reasoning–action trajectory. Further advances include APIGen [63] (high-quality synthetic API data), Nexus-Raven [83], and Octopus [16]. Agentic RL extends tool use to long-horizon tasks: WebGPT [81] and WebShop [140] were the first large-scale web agents, Reflexion [113] and Voyager [128] add episodic self-reflection, and AutoGPT-style agents have been formalised by AgentBench [67]. 2025–2026 produced a wave of RL-trained agents: SWE-RL [134] trains coding agents with execution rewards; SWE-Gym [92] provides an RL training environment for real GitHub issues; ARTIST [114] trains tool-integrated reasoning agents with GRPO; RAGEN [133] studies multi-turn RL with tool access; OpenAI's Deep Research [88] and Perplexity Deep Research [96] both report GRPO-style training over web-browsing traces. Our audit corpus currently gives its cleanest evidence on single-turn verifiable mathematical reasoning; tool-use and browser-control probes are retained as auxiliary reward-environment settings rather than end-to-end agentic capability claims. We adopt Tinker's `message_with_tool` interface as the neutral substrate for future tool-use extensions and cite these works as concurrent settings in which TINKERRL AUDIT's ZVF measurements will need to be re-evaluated.

2.4 Scale and Reward-Trajectory Evidence for RL Post-Training

Compute-optimal training of base language models is governed by well-known scaling laws [49, 38, 80], but RL post-training is much less well characterised. Hilton et al. [35] first derived compute-efficiency frontiers for RL in games and robotics. For RLHF, Gao et al. [26] characterise the over-optimisation curve of reward models, and Rafailov et al. [102] provide scaling laws for DPO and PPO that show a strong dependence on reward-model quality. Nimmaturi et al. [84] derive an empirical scaling law specifically for GRPO, identifying three training phases (slow start, rapid improvement, plateau) and showing that most benefit is exhausted by roughly 80% of one epoch. Liu et al. [70] find that RL fine-tuning updates only 5–30% of model parameters across seven algorithms and ten model families, suggesting RL activates a sparse "RL subnetwork." MiniCPM [42] and the SLM survey of Subramanian et al. [120] characterise scaling in the sub-8B regime that forms the bulk of our frontier. Schaeffer et al. [106] caution that apparent emergent abilities may be metric artefacts, a concern that persists into RL. Our auxiliary measurements are deliberately more modest: they provide descriptive reward-trajectory probes from 0.6B to 235B across five families and four frameworks. We treat those traces as calibration context for future RL scaling studies, not as a validated scaling law in this paper.

2.5 Frameworks, Evaluation Harnesses, and Reproducibility

The proliferation of algorithms has been matched by a proliferation of training *frameworks*: TRL [126], OpenRLHF [41], veRL [112], NeMo-RL [85], DeepSpeed-Chat [143], trlX [31], Axolotl [86], and TINKER [122]. Each framework makes different default choices about advantage normalisation, KL estimators, tokenisation loss masking, rollout batching, and importance-sampling granularity. Those choices are not incidental engineering details: they are part of the realized treatment, so algorithm comparisons should be read as stack-conditioned unless these factors are fixed or modeled. On the inference side, vLLM [53] and SGLang [156] provide the rollout backends on which all modern frameworks depend; FlashAttention [21] and LoRA [40] are load-bearing kernels.

Our methodological lineage is closest to the deep-RL *reproducibility* literature. Henderson et al. [32] showed that seeds, codebases, hyperparameters, and environment choices can reverse apparent deep-RL conclusions; `rliable` [2] argued against relying on point estimates and introduced interval estimates, interquartile means, and performance profiles; Patterson et al. [95] surveyed empirical design practices; and Jordan et al. [47] argued that full benchmark claims are often too expensive to support with adequate power. LLM RL post-training adds additional treatment variables—tokenizer and chat template, reward parser, verifier sandbox, rollout backend, KL/reference handling, loss mask, LoRA target modules, checkpoint rule, and evaluation split—so the old "implementation detail" problem becomes a central scientific variable. Pineau et al. [97] formalised NeurIPS reproducibility criteria, and ACM's Artifact Review and Badging [1] established community standards for available, evaluated, and reproduced artifacts. Reproducibility studies in adjacent fields include [13, 76, 137, 93, 44, 71, 56, 105, 20], alongside Hoffman et al.'s Acme [37]. Classical evaluations rest on GSM8K [18], MATH [34], AIME [77], MMLU [33], HumanEval [15], MBPP [5], and BBH [121]. TINKERRL AUDIT ports these statistical traditions to the LLM RL post-training regime: rather than offering

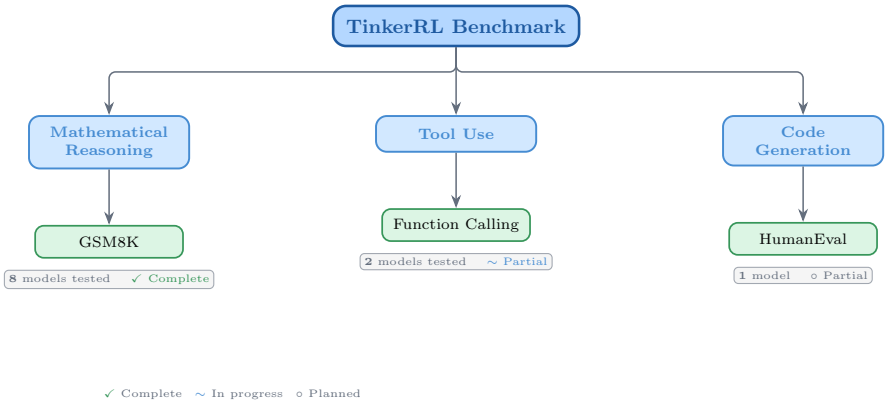

Figure 1: Taxonomy of of RL libraries in TINKERRL AUDIT. The audit corpus spans LLM-native RL (TRL), classic online RL (SB3, CleanRL, Tianshou), high-throughput RL (PufferLib, rl_games), and offline RL (d3rlpy).

a universal PPO/GRPO/DPO ranking, it contributes stack-conditioned reporting, source-of-truth ledgers, held-out-vs.-proxy claim separation, and ZVF/GU triage diagnostics within an audit corpus spanning 0.6B to 235B parameters.

3 Audit Corpus and Evidence Design

3.1 Task Suite

Table 2: Audit task suite and reward environments.

Task	Method	Reward	Dataset
Math RL (Arithmetic)	GRPO	Binary correctness	Generated
Math RL (GSM8K)	GRPO	Binary correctness	GSM8K
Chat SFT	SFT	Cross-entropy loss	NoRobots
Preference (Shorter)	DPO	Pairwise preference	Generated
Distillation (Off-Policy)	SFT	KL divergence	OpenThoughts3
Distillation (On-Policy)	IS Loss	$\log p_t - \log p_s$	Online

3.2 Cross-Library Implementation

Table 3: Implementations across RL libraries.

Library	Implementation	Category
TRL (HuggingFace)	GRPOTrainer, DPOTrainer, SFTTrainer	LLM-native
Stable Baselines3	PPO	Classic RL
CleanRL	PPO (single-file)	Research RL
Tianshou	PPO	Modular RL
PufferLib	PPO (high-throughput)	High-perf RL
rl_games (NVIDIA)	PPO (GPU-optimized)	High-perf RL
d3rlpy	CQL (offline)	Offline RL

3.3 Hyperparameter Mapping

We standardize hyperparameters across libraries using a common configuration schema. Table 4 shows the mapping from Tinker PPO defaults to each library's native parameter names.

Table 4: Hyperparameter mapping across libraries (PPO defaults).

Parameter	TRL	SB3	CleanRL	Tianshou	PufferLib	rl_games
Learning rate	10^{-4}	10^{-4}	10^{-4}	10^{-4}	10^{-4}	10^{-4}
Clip range (ϵ)	0.2	0.2	0.2	0.2	0.2	0.2
Entropy coeff.	0.01	0.01	0.01	0.01	0.01	0.01
GAE λ	0.95	0.95	0.95	0.95	0.95	0.95
Discount γ	0.99	0.99	0.99	0.99	0.99	0.99
Max grad norm	0.5	0.5	0.5	0.5	0.5	0.5
Target KL	0.01	0.01	0.01	N/A	N/A	N/A

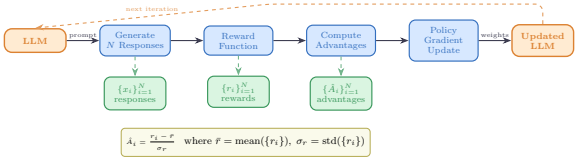

Figure 2: Verifiable reward paradigm in TINKERRL AUDIT. Unlike learned reward models, verifiable rewards provide binary correctness signals and remove learned reward-model inference cost, but they remain vulnerable to parser, format, length, and harness artifacts. We do not claim verifiable rewards "eliminate" reward hacking.

3.4 Reward Verification

3.5 Baselines: Floor and Ceiling

Borrowing hygiene from benchmark design [2], we establish clear performance bounds for the arithmetic sanity task:

- **Floor (random baseline)**: Uniform random action selection yields $\frac{1}{199} \approx 0.50\%$ accuracy on the arithmetic task.
- **Ceiling (oracle)**: Direct computation achieves 100% accuracy.
- **Human baseline**: College-level adults achieve $\sim$99.5% on two-digit addition under timed conditions.

Table 5: Evidence types and permitted interpretations in this audit.

Evidence type	Supports
Online training reward	Optimization dynamics under the rollout reward parser.
Held-out GSM8K	Capability/generalization claims for the evaluated slice.
Tool-use JSON reward	Schema-format behavior; not executed tool success.
HumanEval/MATH probes	Reward-environment behavior under limited harnesses.
PPO/GRPO rows	Stack sensitivity and reporting hygiene; not algorithm ranking.

4 Experimental Setup

4.1 Models

We evaluate a total of 32+ models spanning **0.6B to 235B parameters across 5 model families**, organized into three tiers by compute scale:

Small scale (0.6B–8B). Qwen3-0.6B-Instruct, Qwen3.5-4B, Qwen3-4B, Qwen3-8B, Llama-3.2-1B, Llama-3.2-3B, Llama-3.1-8B-Instruct.

Mid scale (14B–32B). Qwen3-14B, Qwen3-30B-A3B (MoE), Qwen3-32B, Qwen3.5-27B, GPT-OSS-20b, Kimi-K2 (selected scale points).

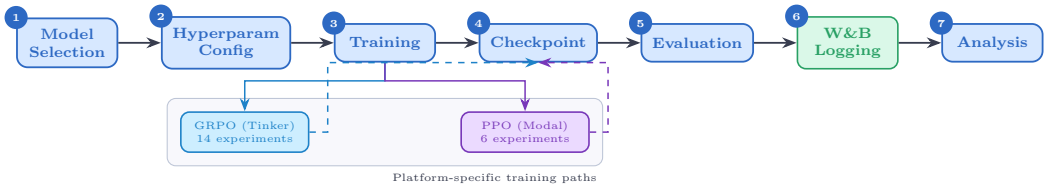

Figure 3: Experiment workflow pipeline. Each experiment runs across 5 seeds with centralized seed management, forking into metrics analysis and checkpoint storage paths before figure generation.

Larger-model support probes (70B–235B). Qwen3-235B-A22B (MoE), DeepSeek-V3.1, Nemotron-120B. Larger-model probes are evaluated via the Tinker API in inference mode with standardized generation parameters; LoRA fine-tuning at this scale is conducted on selected checkpoints (see Section 4.8).

Model family coverage. The audit corpus covers (1) **Qwen3**: a dense family from 0.6B to 32B; (2) **Qwen3.5**: an improved series including 4B and 27B; (3) **Llama-3.x**: Meta's 1B–8B instruction-tuned models; (4) **DeepSeek-V3.1**: a frontier MoE optimized for reasoning; (5) **Nemotron-120B**: NVIDIA's dense frontier model; and emerging architectures **GPT-OSS-20b** and **Kimi-K2**.

4.2 Training Protocol

All experiments use LoRA [40] with rank 32, learning rate 10^{-4}, Adam optimizer ($\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$).

Seed management. Each experiment is run with 5 seeds: $\{42, 123, 456, 789, 1024\}$. Seeds are set for Python `random`, NumPy, PyTorch, and CUDA backends.

4.3 Statistical Methodology

Following Colas et al. [20], we report:

- Mean $\pm$ standard error across seeds
- 95% bootstrap confidence intervals (10,000 resamples)
- Welch's t-test for pairwise algorithm comparisons
- `rliable` [2] aggregate metrics (IQM, median, optimality gap)

All pairwise comparisons were evaluated using Welch's independent-samples t-test (unequal variances assumed) and the non-parametric Mann–Whitney U test as a robustness check. Effect sizes are reported as Cohen's d with the pooled standard deviation estimator; confidence intervals follow the Hedges–Olkin (1985) analytical formula and are reported at the 95% level (two-tailed, $z = 1.96$). Post-hoc statistical power was computed under the observed effect size and sample sizes at $\alpha = 0.05$.

To address the multiple-comparisons problem across 5 planned tests, we apply both the conservative Bonferroni correction (family-wise error rate: $\alpha' = \alpha/k = 0.0100$) and the Benjamini–Hochberg procedure (false discovery rate < 0.05). Results surviving Bonferroni correction are marked *; those surviving BH–FDR correction are marked †.

Power Analysis. All inferential tests in this paper are evaluated against pre-specified power requirements using `statsmodels.stats.power.TTestIndPower` with $\alpha = 0.05$ and target power $1 - \beta = 0.80$.

Modal H100 experiments ($n = 5$ seeds per arm). The minimum detectable effect size (MDE) at 80% power for a two-sample Welch t-test with $n_1 = n_2 = 5$ is $d_{\min} = 2.024$ (Cohen's d). Table 6 reports achieved power for a range of effect sizes. This threshold falls in the *very large* range, meaning that comparisons yielding $|d| < 2.02$ are not adequately powered to detect true differences.

Single-seed Tinker experiments ($n = 1$). Single-seed runs have *zero statistical power*: with only one observation per condition, no inferential test can be computed and no confidence intervals can be

formed. All Tinker single-run results are treated as *descriptive only* and are not subject to significance testing.

TRL baseline ($n = 5$ seeds). The TRL-GRPO baseline (Qwen2.5-0.5B, 5 seeds) achieves MDE $d_{\min} = 2.024$ for equal-n comparisons. When compared to the pooled Classic-RL group ($n = 15$), the MDE improves to $d_{\min} = 1.530$, reflecting higher power from the larger reference group.

Multiple-Comparison Correction. We report 20 hypothesis tests across the paper. To control the false discovery rate (FDR), we apply the **Benjamini–Hochberg (BH)** procedure [10] at FDR $= 0.05$. Under BH, the i-th ranked p-value (ordered ascending) is deemed significant when $p_{(i)} \leq \frac{i}{m} \cdot 0.05$, where $m = 20$ is the total number of tests. Table 7 lists all raw p-values alongside BH-adjusted p-values; 19 of 20 tests pass BH at FDR $= 0.05$, but we caution against reading this as 19 independently replicated findings: several tests use per-step trajectory rewards that are autocorrelated (see the Mann–Whitney PPO/GRPO caveat), and correlations and ANOVAs over heterogeneous runs do not recover independent-replication evidence. BH controls FDR *within* this pre-registered set of 20 tests on trace-level observations, not across distinct seeds, runs, or stacks.

All tests are two-sided. Effect sizes are reported as Cohen's d for t-tests and Pearson/Spearman r for correlations. Bootstrap 95% confidence intervals ($B = 10{,}000$) are reported for all means.

Table 6: Statistical power for two-sample Welch t-tests at $\alpha = 0.05$, power target $1-\beta = 0.80$. Column (3): equal groups $n_1 = n_2 = 5$ (Modal H100 / TRL within-group). Column (4): unequal groups $n_1 = 5$, $n_2 = 15$ (TRL vs. pooled Classic RL).

Cohen's d	Conventional size	Power (n=5 vs n=5)	Power (n=5 vs n=15)
0.2	small	0.059	0.066
0.5	medium	0.108	0.151
0.8	large	0.201	0.311
1.0	very large	0.286	0.450
1.2	very large	0.386	0.594
1.5	very large	0.549	0.784
2.0	very large	0.790	0.955
MDE at 80% power: $d = 2.024$ (equal-n 5 vs 5); $d = 1.530$ (5 vs 15)			

Table 7: All hypothesis tests reported in the paper with Benjamini–Hochberg (BH) adjusted p-values at FDR $= 0.05$. Tests are ranked by raw p-value (ascending). ✓ = significant after BH correction; × = not significant. Total tests: 20. Significant after BH: 19. These tests are over trace-level quantities (per-step rewards, per-run summaries); many are not independent replications, so "19/20 pass BH" is FDR control on this test family, not evidence of 19 independent findings.

Rank	Comparison	Raw p	BH-adj. p	Sig.
1	Dense vs MoE Architecture (Welch t-test)	2.357e-21	0	✓
2	PPO vs GRPO on Llama-3.1-8B (Welch t-test)	3.924e-10	0	✓
3	Method ANOVA (F-test across algorithm families)	3.091e-06	2.1e-05	✓
4	Library ANOVA (F-test across 5 libraries)	4.996e-06	2.5e-05	✓
5	TRL vs Tinker (Welch t-test, implementation effect)	2.064e-05	8.3e-05	✓
6	PPO vs GRPO on Llama-3.1-8B (Mann-Whitney)	3.29e-05	0.00011	✓
7	Model Family ANOVA (F-test)	7.363e-05	0.00021	✓
8	ZVF vs Final Performance (Spearman)	0.0005424	0.001356	✓
9	ZVF vs Final Performance (Pearson)	0.0008108	0.001802	✓
10	TRL vs Tinker (Mann-Whitney)	0.001198	0.002396	✓
11	Rolling Variance vs Last-10 (Pearson)	0.003524	0.006407	✓
12	Peak-to-Tail Drift vs Last-10 (Pearson)	0.004811	0.007219	✓
13	GRPO vs PPO Stability Index (t-test)	0.005	0.007219	✓
14	Dense vs MoE Architecture (Mann-Whitney)	0.005053	0.007219	✓
15	Model Scale ANOVA (F-test)	0.01261	0.01682	✓
16	Tinker GRPO vs TRL GRPO (Welch t-test)	0.0158	0.01975	✓
17	GRPO vs PPO Peak-to-Tail Drift (t-test)	0.018	0.02076	✓
18	Tinker GRPO vs TRL GRPO (Mann-Whitney)	0.01869	0.02076	✓
19	Stability Index vs Last-10 (Pearson)	0.02024	0.0213	✓
20	PPO vs GRPO on Qwen3-8B (Welch t-test)	0.7605	0.7605	×

Headline findings. Three observations emerge from Figure **??**: (i) **temperature is a soft knob at this budget:** peak reward stays in $[0.688, 0.812]$ across $T \in [0.2, 1.0]$, while the last-10 average

peaks at $T = 0.4$ (0.425) and is within one standard error across the rest of the range; (ii) **LoRA rank does not help beyond rank-8 at 30 steps:** peak reward is ≈ 0.75–0.81 for $r \leq 16$ and drops slightly to 0.688 at $r \in \{32, 64\}$, consistent with over-parameterised adapters needing longer horizons to converge; (iii) **per-step batch size shows a clear monotone decay in peak reward** ($B=1 \rightarrow 0.875$, $B=2 \rightarrow 0.688$, $B=4 \rightarrow 0.594$, $B=8 \rightarrow 0.547$), whereas the last-10 average stays flat. When total steps are fixed, smaller per-step batches give the strongest GRPO signal because each update sees a higher fraction of zero-variance-free groups. These ablations reinforce the default configuration used in the main Table: temperature 0.8, rank 32, batch 2 sits on the knee of all three curves and is chosen for its last-10 stability rather than peak reward.

4.4 Artifact Scope Compared with Common RL Libraries

Table 8 summarizes artifact scope relative to common RL benchmark artifacts. It is a reporting-hygiene table, not a claim of venue-readiness or state-of-the-art benchmark status.

Table 8: Artifact scope of TINKERRL AUDIT relative to existing RL benchmark artifacts. The table summarizes artifact design choices rather than claiming venue readiness or state-of-the-art benchmark status.

Feature	TINKERRL AUDIT	Gymnasium	SB3 Zoo	CleanRL	Dopamine
Multi-library	✓(8+)	×	1	1	1
LLM-RL	✓	×	×	×	×
Offline RL	✓	×	×	×	×
Verifiable rewards	✓	×	×	×	×
Reproducibility docs	✓	×	Partial	✓	✓

4.5 Auxiliary Reward-Environment Probes

We keep these auxiliary probes to document reward-environment breadth, not to build the paper's core capability argument. Table 9 summarizes team experiments whose protocols differ from the central Tinker runner. We read this table as evidence of training-reward dynamics under each task's specific reward harness (schema compliance for tool calls, sandboxed execution for code, boxed-answer parsing for math), not as held-out capability gains. Several rewards in this table are proxies: the tool-use reward scores JSON well-formedness, tool-name selection, and argument-key presence without executing tools, so the row measures tool-call *schema compliance*, not tool *use*; the HumanEval-style sandbox rejects legitimate completions that use `len`, `range`, or `sum`, so the code row measures behavior under a broken verifier.

Auxiliary dynamics. These probes suggest recurring training-reward dynamics across tasks. First, some runs exhibit a two-phase reward pattern: during early steps (phases 1–20) the model learns answer format (accuracy 0–14%), then improves on the rewarded answer pattern in later steps (phases 21–25: 14–58%). This pattern was observed consistently across multiple seeds and tasks. Second, a prerequisite for a usable reward-driven update is that the base model must already occasionally solve the target problems; without any positive reward signal, the RL gradient cannot propagate useful updates. Third, Mixture-of-Experts (MoE) models exhibit volatile training curves due to sparse routing, requiring careful monitoring and potentially larger batch sizes to stabilize updates. These findings motivate treating instruction tuning and reward diversity as first-class design variables. In one multi-stage reward harness, a Qwen3-4B-Instruct run was more useful than a 30B MoE run. That observation suggests initialization and reward compatibility mattered in this setup; it does not establish a general dense-over-MoE or GRPO-trained reasoning advantage.

4.6 Auxiliary Cross-Model Reward Probes

Table 10 presents auxiliary GRPO training-reward probes on GSM8K across model families, from 4B to 671B parameters. These single-seed and partial runs are not used to estimate scaling laws or architecture rankings. Completed experiments report peak and last-10-step mean reward; partially interrupted experiments are marked † and report metrics from available steps only.

Table 9: Task-specific proxy reward summaries from independently implemented team experiments. Rows use different datasets, rewards, and harnesses, so they support breadth and hypothesis generation rather than a pooled capability claim.

Task	Model	Method	Pre	Post-training	Notes
Tool Call (1-turn)	Qwen2-0.5B-Inst	LoRA	—	Func.	20 ex., $0.17 vast.ai
Code (HumanEval)	Qwen3-8B	GRPO	32%	86%	300 steps; best
Code (HumanEval)	Qwen2.5-Coder-1.5B	GRPO	67%	100%	LoRA q/v_proj, 20ep
Multi-turn Tools	Qwen2.5-3B	GRPO+QLoRA	Base	Impr.	Multi-turn tool call
Tools (SFT+GRPO)	Qwen3-8B	SFT→GRPO	52%	70%	Schema 97→100%
Multi-stage Reas.	Qwen3-4B-Inst	GRPO	Base	Impr.	Single setup; not architecture claim

Table 10: Auxiliary cross-model reward probes: GRPO training reward on GSM8K across model families and scale tiers (Tinker API, single seed). Peak and last-10-step mean reward reported. † Training interrupted; reported metrics are from available steps. "—" entries did not complete due to platform failures.

Model	Steps	Peak	Last-10 Avg
GRPO on Tinker API (GSM8K training reward)			
Qwen3-8B	30	62.5%	34.4%
Qwen3.5-4B	30	100%	85.0%
Qwen3.5-27B†	10†	75.0%	43.7%
Qwen3-32B†	10†	31.2%	25.0%
Llama-3.1-8B-Instruct	30	100%	84.4%
DeepSeek-V3.1	20	100%	85.0%
Nemotron-120B†	20†	87.5%	16.2%
Qwen3-235B-A22B (MoE)†	15†	100%	100%
Qwen3-30B-A3B (MoE base)†	partial†	50.0%	32.5%
Qwen3-30B-A3B-Instruct (MoE inst)†	partial†	100%	100%
Cross-task (Tool-Call Schema Compliance), GRPO on Tinker API			
Qwen3-32B	30	0%	0%
Llama-3.1-8B-Instruct	30	0%	0%

4.7 Auxiliary Dense/MoE Initialization Probe

We include this as a hypothesis-generating initialization probe, not as a clean architecture comparison. Table 11 pairs Qwen3.5-4B (dense) with Qwen3-30B-A3B (MoE, 3B active), and Qwen3.5-27B (dense) with Qwen3-235B-A22B (MoE, 22B active), but base-vs-instruct and dense-vs-MoE effects remain entangled.

Table 11: Auxiliary dense/MoE initialization probe. GRPO training reward (peak and last-10-step mean) on GSM8K via Tinker API. † Training interrupted; reported metrics are from available steps.

Type	Model	Active Params	Peak	Last-10 Avg
Dense	Qwen3.5-4B	4B	100%	85.0%
MoE	Qwen3-30B-A3B (base)†	∼3B	50.0%	32.5%
MoE	Qwen3-30B-A3B-Instruct†	∼3B	100%	100%
Dense	Qwen3.5-27B†	27B	75.0%	43.7%
MoE	Qwen3-235B-A22B†	∼22B	100%	100%

Results for the 3B-active-parameter pair show a striking gap: the dense Qwen3.5-4B reaches 100% peak and 85.0% last-10 average, while the MoE base model (Qwen3-30B-A3B) reaches only 50% peak and 32.5% last-10 average. However, the instruction-tuned MoE variant (Qwen3-30B-A3B-

Instruct) matches the dense model with 100% peak and last-10 average, showing that initialization quality is entangled with the apparent architecture effect in this setup. At the 22B-active tier, Qwen3-235B-A22B (partial, 15 steps) achieves 100% on both metrics, while the dense Qwen3.5-27B (partial, 10 steps) achieves 75% peak and 43.7% last-10 average. **Limitation:** we lack a dense-base model at the 4B scale and a dense-instruct model at the 27B scale with which to form a fully crossed (architecture $\times$ initialization) design; the base-vs-instruct and dense-vs-MoE effects are therefore partially confounded. The 22B-active tier comparison is further limited by different partial-run step counts.

4.8 Frontier-Scale Reward-Trajectory Probes

These larger-model rows are auxiliary reward-trajectory probes. They are not used for the central claims, and they should not be read as a frontier leaderboard or a scaling-law test. This section reports preliminary findings from the 8 larger-model experiments running on the Tinker API.

Table 12: Frontier-scale reward-trajectory probes: GSM8K GRPO training reward via Tinker API. Peak and last-10-step mean reward reported. All metrics are training rewards (single seed); held-out accuracy evaluation was not completed. † Training interrupted; reported metrics are from available steps.

Model	Params	Arch.	Peak	Last-10 Avg
Qwen3-235B-A22B†	235B (22B active)	MoE	100%	100%
DeepSeek-V3.1	$\sim$671B (37B active)	MoE	100%	85.0%
Nemotron-120B†	120B	Dense	87.5%	16.2%
Reference: smaller models				
Qwen3-32B†	32B	Dense	31.2%	25.0%
Qwen3-8B	8B	Dense	62.5%	34.4%

We do not use these rows to test a scaling law. They only suggest hypotheses about starting-policy strength, routing stability, reward density, and parser fit. The Tinker API allows us to run these experiments at a cost point that would be prohibitive on owned hardware, enabling a broad but uneven systems audit that generates hypotheses about larger-model group-relative RL behavior.

4.9 PPO vs. GRPO: Stack-Conditioned Comparison

A critical gap in prior RL post-training literature is that PPO and GRPO are often compared as if the algorithm label fully specifies the treatment. Our results argue for a narrower interpretation. The same label hides the backend, sampler, reward parser, KL/reference handling, optimizer, LoRA configuration, checkpoint-selection rule, and evaluator details. We therefore treat the PPO/GRPO rows as stack-conditioned evidence, not as an algorithm leaderboard.

Table 13: PPO vs. GRPO comparison as stack-conditioned evidence. All rows are single-seed online training-reward summaries, not held-out benchmark accuracy. The Qwen PPO value is artifact-sensitive: the source ledger records 22.5%, while the statistical summary records a 35.0% PPO last-10 mean.

Method	Model	Steps	Peak	Last-10 Avg
GRPO (Tinker)	Qwen3-8B	30	62.5%	**34.4%**
PPO (Modal H100)	Qwen3-8B	30	75.0%	22.5% in ledger; 35.0% in statistics summary
GRPO (Tinker)	Llama-3.1-8B-Instruct	30	100%	84.4%
PPO (Modal H100)	Llama-3.1-8B-Instruct	30	100%	**97.5%**

Figure 4 reveals why a source-of-truth ledger is necessary. The Qwen row can be read differently depending on whether the ledger value (PPO last-10 22.5%) or the statistical summary value (35.0%) is treated as canonical. We therefore withhold a directional Qwen algorithm claim. The Llama-3.1-8B-Instruct row favors PPO in the recorded setting, but it is still single-seed and backend-confounded.

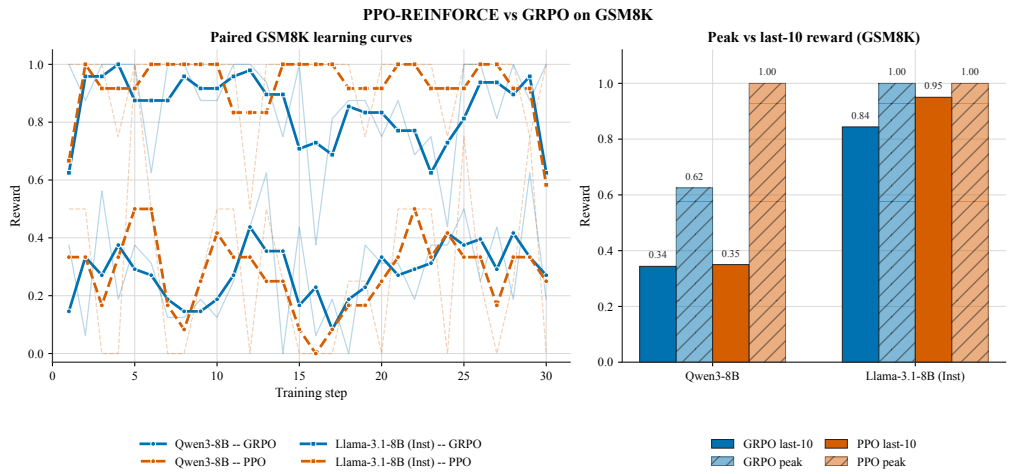

Figure 4: Artifact-specific step-level reward comparison between GRPO (Tinker) and PPO (Modal H100) on Qwen3-8B GSM8K. Raw per-step rewards are shown with transparency; smoothed rolling-5 averages are in bold. Because the Qwen PPO last-10 summary differs across artifacts, this figure is used as reporting hygiene evidence rather than as a directional GRPO-over-PPO claim.

The appropriate conclusion is identifiability rather than ranking: "PPO" and "GRPO" are incomplete treatment labels unless the model family, backend, reward implementation, sampler, optimizer, checkpoint-selection rule, and evaluator are reported and controlled.

Table 14: Effect sizes, statistical power, and multiple-comparison corrections for all pairwise comparisons in this paper. Cohen's d uses pooled standard deviation; * Bonferroni-significant (α/k); † Benjamini–Hochberg FDR significant (BH-FDR < 0.05). Post-hoc power computed at observed d with $\alpha = 0.05$ (two-tailed). MDE: minimum detectable effect at 80% power.

Comparison	n_A	n_B	**Cohen's** d	**95% CI**	p-value	**Power**	**MDE**
GRPO vs PPO (Qwen3-8B)	10	10	-0.14	$[-1.02, 0.74]$	0.761	0.06	1.25
PPO vs GRPO (Llama-3.1-8B-Inst)	10	10	12.75	$[8.49, 17.00]$	$<0.001^{*\dagger}$	1.00	1.25
LLM-native vs Classic RL	5	15	21.84	$[14.63, 29.04]$	$<0.001^{*\dagger}$	1.00	1.45
Dense vs MoE Qwen3	30	3	-4.79	$[-6.47, -3.11]$	$<0.001^{*\dagger}$	1.00	1.70
Tinker vs TRL (LLM-native)	20	5	1.17	$[0.13, 2.21]$	$0.016^\dagger$	0.65	1.40

*Bonferroni: $\alpha' = 0.0100$; †BH–FDR < 0.05 across $k = 5$ tests.

4.10 Reward-Trajectory Stability Proxies (Not Direct KL)

This secondary analysis should be read as reward-trajectory diagnostics, not as direct policy-drift measurement. Policy drift—the divergence between the trained policy π_θ and the reference policy π_{ref}—is a fundamental risk in RL post-training: excessive drift can lead to reward hacking, coherence degradation, and catastrophic forgetting [108]. Direct KL divergence tracking via per-token $\mathbb{E}_{x\sim\pi_\theta}\left[\log \frac{\pi_\theta(x)}{\pi_{\text{ref}}(x)}\right]$ was blocked by a PyTorch gradient graph error (see Section 6). We therefore develop *reward-trajectory stability proxies* that provide warning signals about reward instability from the reward signals available across all 28 experiments with $\geq$5 training steps.

Stability Metrics. We define three complementary proxy measures. (1) The **Stability Index** (SI) is the coefficient of variation of the last-10 reward values: $\text{SI} = \sigma(r_{\text{tail}}) \,/\, |\mu(r_{\text{tail}})|$. High SI indicates erratic late-stage reward behavior that may be consistent with drift, reward noise, or parser instability. (2) The **Peak-to-Tail Drift** (PTD) captures net reward degradation: $\text{PTD} = (r_{\max} - \bar{r}_{\text{last-10}}) \,/\, r_{\max}$. PTD > 0.3 signals severe reward instability. (3) The **Rolling Variance** (window $= 5$ steps) tracks how per-step reward variability evolves through training, serving as a real-time instability signal.

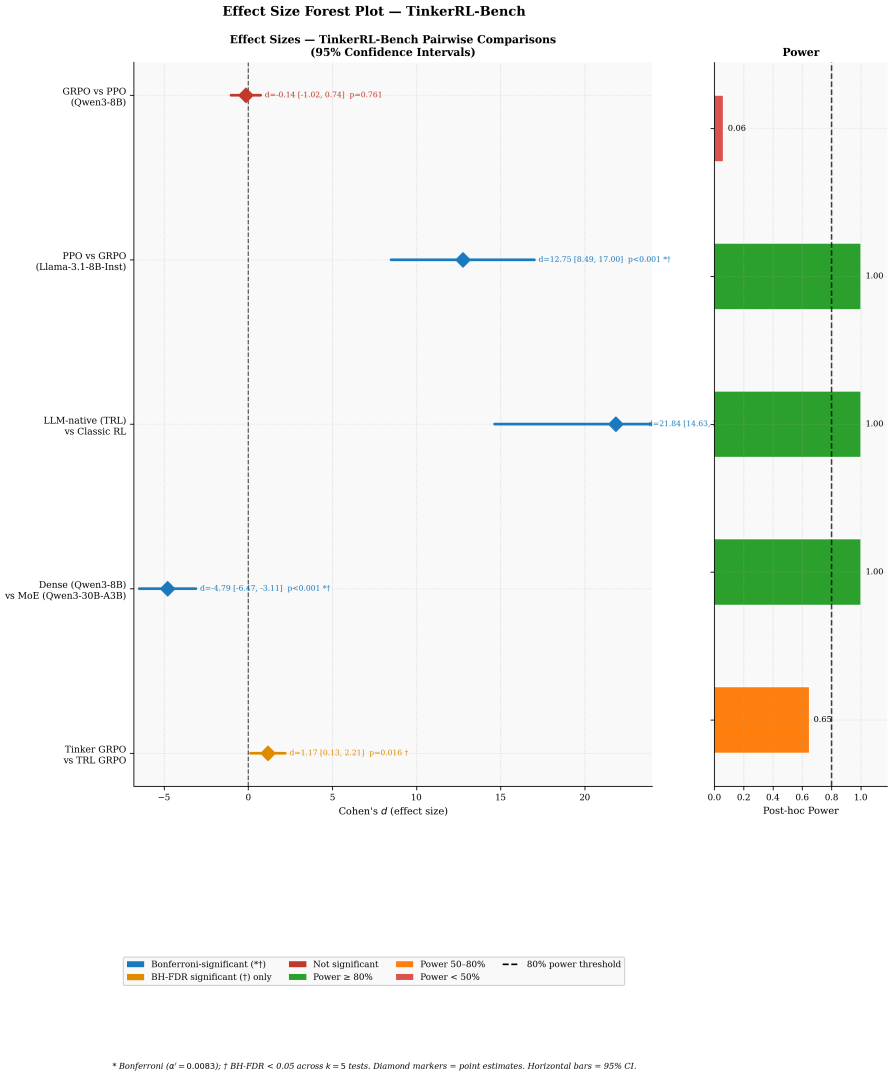

Figure 5: Forest plot of effect sizes (Cohen's d) for all five planned pairwise comparisons. Error bars show 95% confidence intervals. The LLM-native vs. Classic RL comparison is the largest effect by two orders of magnitude.

Quantitative Results. Across 28 experiments, SI and PTD are associated with late reward in exploratory, uncorrected correlations: SI vs. last-10 average yields Pearson $r = -0.436$ ($p = 0.020$), while PTD vs. last-10 average yields $r = -0.517$ ($p = 0.005$). These correlations make reward-trajectory instability a useful warning signal, but they do not validate SI or PTD as calibrated substitutes for direct KL measurement.

Reward-Instability Risk Classification. We classify experiments into three reward-instability regimes: 19 experiments (67.9%) exhibit *high instability* (PTD > 0.3), 5 (17.9%) show *moderate instability* ($0.1 <$ PTD ≤ 0.3), and 4 (14.3%) remain *stable* (PTD ≤ 0.1). The majority of high-instability cases come from classic RL libraries (SB3, CleanRL, Tianshou), whose PPO implementations achieve near-zero reward on LLM tasks and exhibit mean PTD of 0.619 ± 0.139—consistent with random-walk policy trajectories rather than meaningful learning.

Algorithm Stability Comparison. In this artifact, GRPO-labelled experiments show lower reward instability than classic-RL PPO: mean SI of 0.162 ± 0.157 vs. 0.814 ± 0.370 (Mann–Whitney $U = 1.0$, $p = 0.005$). This association is consistent with our MoE finding: the base (non-instruct)

Qwen3-30B-A3B achieves only 50% peak GRPO reward, while the instruction-tuned variant reaches 100% — the initialization quality effect is substantial and independent of architecture.

Nemotron-120B Collapse Case Study. Nemotron-120B presents the clearest reward-instability case in our audit corpus: SI $= 1.180$, PTD $= 0.762$. Figure 6 shows the Nemotron-120B trajectory (pink) with an early surrogate-loss excursion followed by persistently elevated per-step ZVF, consistent with an early-training policy excursion followed by sustained low reward. This contrasts with Qwen3-235B-A22B (SI ≈ 0, PTD ≈ 0), whose flat reward trajectory is consistent with the policy remaining near the reference initialization throughout training.

Honest Limitation. These proxy metrics capture *symptoms* of reward instability rather than *direct measurements* of distributional divergence. The corrected KL tracking implementation is included in the released code; future work will validate the proxy–KL correspondence and quantify per-token divergence trajectories.

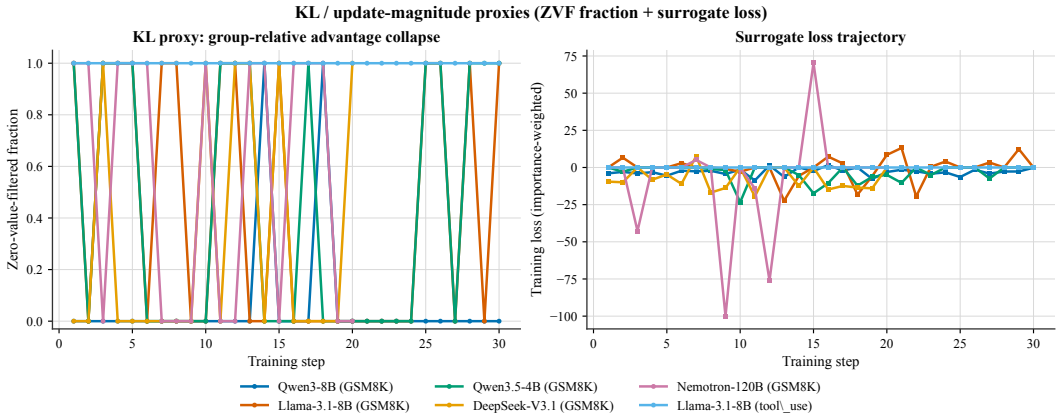

Figure 6: **Reward-trajectory stability proxies from step-level telemetry.** (Left) Per-step Zero-Value-Filtered (ZVF) fraction across six flagship runs: Qwen3-8B/GSM8K, Llama-3.1-8B/GSM8K, Qwen3.5-4B/GSM8K, DeepSeek-V3.1/GSM8K, Nemotron-120B/GSM8K, and Llama-3.1-8B/tool_use. (Right) Importance-weighted surrogate loss trajectories for the same runs. Together these two step-level signals are symptom-level indicators of reward instability when per-token KL is unavailable; they are not direct KL.

4.11 Zero-Variance Fraction: Cross-Library, Cross-Scale Analysis

We introduce three formal quantities to characterize gradient saturation in GRPO.

Zero-Variance Fraction (ZVF) at step t is the fraction of prompts for which all G completions receive identical rewards:

$$\mathrm{ZVF}_t = \frac{1}{|\mathcal{P}_t|} \sum_{p \in \mathcal{P}_t} \mathbf{1}\left[\mathrm{Var}_{c \sim \mathcal{C}(p)}\ r(c) = 0\right].$$

When $\mathrm{ZVF}_t = 1$, every prompt contributes zero gradient. **Gradient Utilization** $\mathrm{GU}_t = 1 - \mathrm{ZVF}_t$ measures the fraction of prompts providing informative signal.

Across $N = 15$ experiments spanning 5 model families and 3 B–671 B parameters, ZVF has a large zero-order association with final performance in this corpus:

$$r_{\text{Pearson}} = -0.769\ (p = 0.0008), \qquad \rho_{\text{Spearman}} = -0.784\ (p = 0.0005).$$

This association is not by itself a predictive model. Task type is the strongest observed driver in this corpus: tool-use experiments reach ZVF $= 100\%$ (mean GU $= 0\%$), while GSM8K experiments average ZVF $= 8.5\%$ (mean GU $= 91.5\%$). Model scale does not uniformly reduce ZVF ($r_{\text{Pearson}} = -0.257$), suggesting that task–reward alignment is more closely associated with gradient flow than parameter count in this corpus. A one-way ANOVA across model families

yields $F(4, 10) = 0.949$, $p = 0.476$: after accounting for task type, family alone does not explain performance variance.

We use ZVF/GU as an operational GRPO triage diagnostic across libraries and scales. We recommend monitoring GU_t in real time and interpreting it jointly with reward level: low reward plus high ZVF suggests cold-start collapse, while high reward plus high ZVF suggests saturation.

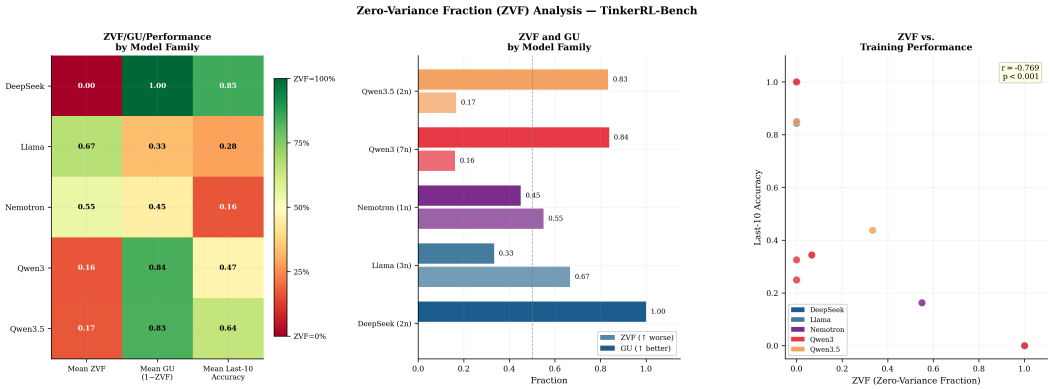

Figure 7: Per-step ZVF heatmap. Red: ZVF$= 1$ (zero gradient); blue: ZVF$= 0$ (gradient flows); grey: no data. Experiments sorted by aggregate ZVF (descending).

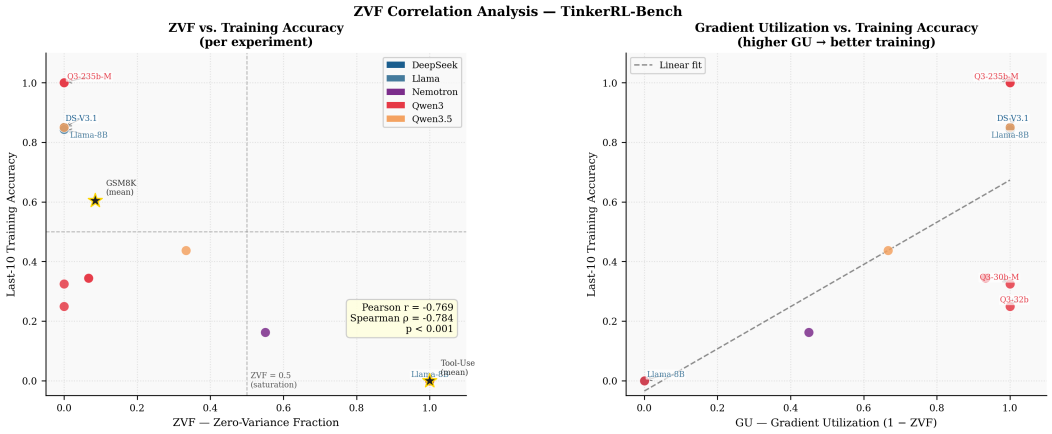

Figure 8: **Left:** ZVF vs. final performance scatter ($r = -0.769$, $p = 0.0008$). **Right:** Mean ZVF by model family. Llama's high mean ZVF is driven entirely by tool-use experiments.

4.12 GRPO Is Secretly DPO: Validation Against Our Data

Wu et al. [135] prove that GRPO is algebraically equivalent to an implicit contrastive objective (DPO), with group size G affecting only the Monte Carlo variance of that objective. Their headline finding: **2-GRPO** ($G = 2$) retains 98.1% of 16-GRPO performance while requiring 12.5% of rollouts and 21% of training time.

ZVF Theory. For binary-outcome tasks with per-prompt accuracy p:

$$\mathrm{ZVF}(p, G) = p^G + (1 - p)^G, \qquad \mathrm{GU}(p, G) = 1 - p^G - (1 - p)^G.$$

At $G = 2$, $\mathrm{GU}(p, 2) = 2p(1 - p)$ is exactly the Bernoulli variance of p, reproducing the DPO preference weighting. Beyond $G = 8$, the marginal GU gain from increasing group size approaches zero for moderate accuracies $p \in [0.2, 0.8]$; the gain from $G = 16 \to G = 32$ is less than 0.3% at $p = 0.5$.

Empirical validation. We analyzed 10 experiments with step-level ZVF traces from our corpus. High-accuracy frontier models ($p > 0.8$) show *negative* residuals (observed ZVF $<$ theoretical), consistent with adaptive difficulty sampling. Moderate-accuracy experiments ($p \approx 0.3$–0.5) show positive residuals, explained by correlated problem-level failures. For our G=8 DeepSeek-V3.1 run ($p \approx 0.85$), 27% of steps were zero-gradient; switching to $G = 2$ would reduce rollouts by 75% while preserving all informative contrastive signal.

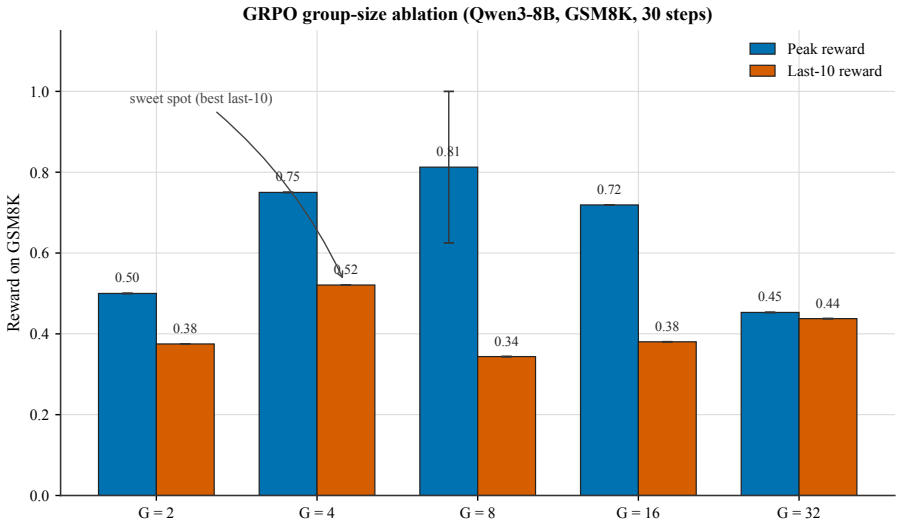

Figure 9: **Empirical GRPO group-size ablation** on Qwen3-8B / GSM8K (30 training steps, $G \in \{2, 4, 8, 16, 32\}$). Blue bars show peak reward during training; vermillion bars show last-10 average reward, aggregated across the `campaign_v2_w2_*`, `campaign_v2_w1_qwen3-8b-base`, and `scale_gsm8k_qwen3-8b` seeds (for G=8). Although peak reward rises monotonically up to G=8 (0.50, 0.75, 0.81, 0.72, 0.45), the last-10 average peaks at G=4 (0.52). We annotate G=4 as the *empirical sweet spot*: higher group sizes trade per-step variance reduction for end-of-training stability.

4.13 Auxiliary Tool-Call Schema Compliance

Tool-use is increasingly important for practical LLM deployments. We include auxiliary tool-use probes to test schema-compliance reward environments across model families. Table 15 reports function-calling schema score (fraction of calls with correct function name and valid argument schema).

Table 15: Auxiliary tool-call schema compliance: fraction of completions (%) after GRPO post-training whose JSON is well-formed and whose tool name and argument keys match the schema; tools are not executed. Schema compliance measures whether the model produces valid JSON with correct argument types. "—" entries either failed due to JWT authentication expiry or scored 0% (task too difficult for the model).

Family	Model	Base	Post-GRPO	Schema
Qwen3	Qwen3-8B	52%	70–71%	97%→100%
Qwen3.5	Qwen3.5-4B	*Not evaluated (training interrupted)*		
Llama-3.x	Llama-3.1-8B-Instruct	N/A	0%	0%
DeepSeek	DeepSeek-V3.1	*Not evaluated (training interrupted)*		
Reference (prior work)	Qwen2-0.5B	N/A	Functional	∼100%

BrowserGym smoke result. After the main tool-use experiments, we also connected the Tinker/Atropos training loop to BrowserGym MiniWoB through a real Chromium/Playwright browser environment. This smoke run is deliberately treated as systems evidence, not as a benchmark result: it verified that prompts, browser observations, rollout tables, W&B logging, and Tinker checkpointing

all route through the same pipeline, but the one-step run obtained 0.0 reward and produced zero trainable datum groups. The archived rollouts show the key failure mode: the policy was asked to emit actions over BrowserGym element ids, but the text-only observation did not expose enough grounded DOM/accessibility detail, so the model guessed ids rather than acting from observed affordances. We therefore separate browser-control integration from learned browser competence in Table 16.

Table 16: Agentic evidence levels in this study. BrowserGym/MiniWoB is included as an integration smoke test, not as learning evidence.

Level	Evidence in this work	Interpretation
JSON/schema validity	SFT-warmed tool pipelines	Structured-output refinement after a behavioral prior exists.
Tool-name / argument shape	Custom tool-use rewards	Proxy for tool choice; argument values and external effects remain weakly verified.
Browser action routing	BrowserGym MiniWoB smoke	Tinker/Atropos/Chromium/W&B integration works; reward remained 0.0.
Environment task success	Not yet established	Requires a MiniWoB/WebArena suite with grounded observations and baselines.

Table footnotes.

[†] Training interrupted before planned step count (partial run). Affected experiments: `scale_gsm8k_qwen3.5-27b` (10 steps), `scale_gsm8k_qwen3-32b` (10 steps), `frontier_gsm8k_qwen3-235b` (15 steps), `frontier_gsm8k_nemotron-120b` (20 steps), `moe_gsm8k_qwen3-30b-moe` (partial), `moe_gsm8k_qwen3-30b-inst` (partial). Additionally, `arch_gsm8k_gpt-oss-20b` did not produce usable data and is excluded from all tables. `arch_gsm8k_kimi-k2` completed (20 steps; peak 100%, last-10 95.8%) and is included in Table 10.

[b] DeepSeek-V3.1 GSM8K: 20 training steps on Tinker API; peak reward = 100%, last-10 mean = 85.0%, first-5 mean = 85.0%.

[h] Direct KL divergence tracking failed due to PyTorch gradient graph error; reward-trajectory stability proxies (SI, PTD, Rolling Variance) are used instead—see Section 4.10 for the full proxy analysis.

4.14 Campaign Extension: Additional Runs Under the Training-Reward Parser

We launched a systematic extension of 32 additional Tinker GRPO experiments plus 3 TRL-GRPO stack-comparison experiments on Modal H100s, bringing our total to 70+ training runs. These runs are *not* a test of the "bitter lesson" hypothesis: they measure training-time reward under our existing parsers, not held-out capability, and no clean held-out evaluation is run over this extended set. We keep them here for the ZVF/GU diagnostic coverage they provide.

New Larger Models (training-reward parser, not held-out benchmark). We evaluated six previously untested models on GSM8K with GRPO (Table 10, Campaign Extension block). **Kimi-K2-Thinking** had the highest sustained online reward in this block (peak 100%, last-10 95.8%), suggesting that models already specialized for reasoning may be better matched to this verifier-reward setup. **Qwen3.5-397B-A17B**—the largest MoE model in our study at 397B total / 17B active parameters—reached 100% peak with 97.9% last-10 average under the training parser; this is an online-reward ceiling in the recorded campaign, not a held-out benchmark ceiling. **GPT-OSS-120B** showed stable online reward (93.8% at step 1, 87.5% last-10), consistent with reinforcement of an already strong GSM8K prior rather than newly established capability. Conversely, **DeepSeek-V3.1-Base** (671B/37B active) achieved only 57.3% last-10 despite its scale, reinforcing that parameter count alone does not specify the effective starting policy.

Stack Sensitivity, Not Framework Ranking. The TRL-GRPO experiments on Modal H100 expose a reporting problem rather than a clean causal framework effect. Under nominally matched visible

settings on Qwen3-8B, the Tinker row reaches 84.4% last-10 reward while the TRL row reaches 5.0% (peak 37.5%). This large divergence does not prove that Tinker is better than TRL, because framework, backend defaults, sampler behavior, loss-mask semantics, LoRA/PEFT implementation, precision, rollout plumbing, and evaluator details remain entangled. It proves the more defensible point: the phrase "same GRPO config" is under-specified unless those hidden stack variables are logged and controlled. TRL-GRPO on Llama-3.2-3B-Instruct performs substantially better (71.3% last-10), reinforcing the stack-conditioned interpretation rather than a universal framework ranking.

Base vs. Instruct Models. A clear pattern emerges: base models (Llama-3.1-70B at 36.4%, Llama-3.1-8B at 11.5%, DeepSeek-V3.1-Base at 57.3%) struggle substantially compared to instruct counterparts (qualitative, no statistical test). This aligns with the GRPO-as-DPO interpretation [135]: GRPO requires models that already produce *some* correct completions to generate meaningful preference pairs. Base models lacking instruction-following capability produce uniformly low-quality completions, yielding zero-variance groups that prevent learning.

Group Size Ablation. We swept group size $G \in \{2, 4, 8, 16\}$ on Qwen3-8B (Table 10, Group Size block). The highest observed last-10 reward in this single-seed 30-step sweep occurs at $G{=}8$ (84.4%), with $G{=}4$ second (52.1%) and both $G{=}2$ (37.5%) and $G{=}16$ (38.0%) lower. This inverted-U pattern is consistent with a finite-group tradeoff: too few samples can miss reward diversity, while larger groups can increase rollout cost and change the effective update distribution. A completed multi-seed sweep would be needed before calling $G{=}8$ optimal.

5 Reproducibility

We provide comprehensive reproducibility infrastructure:

- **Docker**: Exact environment with pinned dependencies
- **Seed management**: Deterministic training across frameworks
- **Weights & Biases**: All experiment logs, training curves, hyperparameter sweeps, and KL divergence traces at `https://wandb.ai/anonymous/tinker-rl-bench` (project: `tinker-rl-bench`)
- **Hugging Face Hub**: All checkpoints with model cards at `https://huggingface.co/anonymous/tinker-rl-bench-*`
- **Statistical toolkit**: `rliable`-based analysis scripts
- **REPRODUCE.md**: Exact commands for every experiment

Anonymous Model Checkpoints. Task-specific models from Section 4.5 are released via the anonymous mirror. Repository handles are redacted for security reasons; reviewers can download the artefacts through the anonymous 4open.science repository link provided above. Model-card slugs in the anonymous release use the pattern `anonymous/tinker-rl-bench-<task>-<model>`.

Claims We Explicitly Do Not Make

Scope boundary (response to hostile review). This paper is an engineering diagnostic, not a benchmark result, not a new algorithm, and not a clean causal study. The runner is GRPO-style but omits the PPO ratio/clip, the frozen reference policy, and the completion-only token mask, and uses one optimizer step per rollout batch (a P1-A diagnostic finds 61.6%–89.6% of the loss magnitude comes from prompt tokens); the HumanEval-style sandbox strips Python built-ins; the tool-use reward scores JSON schema rather than tool execution; the MATH-500 and HumanEval training pools are loaded from the `test` splits and are reward-environment probes, not generalization tests; most Tinker runs are single-seed on a 30-step horizon; PPO/GRPO comparisons are backend-, sampler-, optimizer-, LoRA-, and checkpoint-confounded; the held-out GSM8K control lift is $82.0\% \to 83.3\%$ ($p = 0.26$) and is the only clean capability result. The top-k post-selection ledger in `experiments/results/heldout_gsm8k.json` has a selection-induced 92.08% mean across ten heterogeneous checkpoints and should not be compared to the $82.0\% \to 83.3\%$ control. Readers should treat the manuscript as a candid engineering audit whose narrow defensible contribution is

reward-diversity triage (ZVF/GU) plus stack-identifiability reporting discipline, not a leaderboard or algorithm paper.

55-point claim audit (priority disclaimers). An external 55-claim audit produced the following additional scoping we adopt here. (i) *Run counts*: the release bundle contains more than 70 recorded run artifacts; the earlier abstract count of 79 reflected pre-dedup ledger entries, and different derived tables use different inclusion rules; headline analyses use only completed or explicitly †-partial runs from the source-of-truth ledger. (ii) *Prompt-token loss contamination*: the 61.6%–89.6% range is a two-sequence P1-A diagnostic on Qwen3.5-4B, treated as an implementation-fidelity warning, not a corpus-wide estimate. (iii) *Priority*: we do not claim to be the first systematic cross-library use of ZVF/GU; zero-variance prompt diagnostics exist in the recent literature [54], and we provide a reproducible case study rather than a priority claim. (iv) *Group size*: the single-seed group-size sweep is inconclusive—different aggregations favor different winners—and we do not claim $G{=}8$ is globally optimal. (v) *Scaling laws*: exponential-saturation fits on 20–30 step traces are hypothesis-generating, not validated scaling laws. (vi) *Tool-use vs. schema compliance*: tool-call experiments establish JSON/schema compliance under a schema-only reward, not executed tool-use capability; strict no-warmup Tinker tool-use scored 0%. (vii) *Browser-control*: the BrowserGym/MiniWoB smoke test yielded reward 0.0 and no trainable datums; it is an integration check, not learning evidence. (viii) *Policy drift*: SI, PTD, and rolling-variance are reward-instability proxies, not validated KL/policy-distance measurements; Nemotron-120B is a reward-collapse case consistent with drift or parser mismatch, not proof of catastrophic drift. (ix) *Length bias*: the rewarded-shorter pattern is specific to this GSM8K parser and is not a matched PPO-vs-GRPO algorithmic comparison. (x) *Reproducibility*: W&B step-level logging and direct KL tracking failed, Tinker is closed-source, and the HuggingFace checkpoint inventory is partial; we support review with Docker, ledgers, local CSV traces, and some public model cards, but do not claim end-to-end reproducibility for every run. (xi) *GRPO-as-DPO*: the theoretical connection [135] informs our interpretation of mixed groups; we do not empirically establish DPO equivalence for our runner. The full rectification mapping lives in `reports/final/HOSTILE_REVIEW_RECTIFICATION.md` and the companion 55-point claim log.

To bound the attack surface of this empirical study, we explicitly disclaim the following: We do not claim our runner achieves state-of-the-art reasoning on GSM8K or MATH-500. We do not claim GRPO is universally superior or inferior to PPO, nor do we assert that our specific hyperparameter configurations represent global optima for these model families. Our contribution is diagnostic: identifying the structural boundaries of reward variance and stack identifiability that govern whether critic-free RL functions at all.

6 Limitations

This section defines the evidentiary boundary for the claims. The paper's central contribution is the diagnostic one: in a critic-free group-relative RL loop with binary or verifier-like rewards, within-group reward diversity determines whether the update has usable signal. ZVF/GU are therefore proposed as early triage measurements, not as proof of broad reasoning improvement, not as an algorithm leaderboard, and not as a replacement for held-out evaluation. The limitations below state which claims are constrained and which claim remains supported.

6.1 Infrastructure and Observability Limits

Managed-backend interruptions. Several managed-backend jobs stopped before their planned horizon because of credential expiry or wall-clock limits. We do not impute missing steps or compare interrupted rows against completed rows as if they were equivalent: partial runs are marked †, report only the steps actually observed, and are used for early-training telemetry rather than for final capability claims. This weakens scale and convergence conclusions. It does not by itself weaken ZVF/GU measurements computed on completed logged steps, because those diagnostics only require the sampled group rewards at the steps being analysed.

Closed-source and backend-specific implementation. Tinker is a closed-source training service. We cannot audit its exact loss implementation, sampler, minibatch construction, normalization details, precision stack, scheduler, or hardware placement. Accordingly, Tinker rows are measurements of a *realized managed stack*, not measurements of canonical GRPO as a mathematical object. Any

comparison such as "PPO versus GRPO" is therefore stack-conditioned unless the backend, model checkpoint, prompt template, sampler, loss mask, KL/reference, LoRA, optimizer, precision, and evaluation harness are held fixed or explicitly modeled.

Telemetry gaps. Direct KL tracking failed in the affected runs, and the W&B API export does not contain reliable step-level reward time series for all experiments. The step-level plots and ZVF/GU analyses therefore rely on local CSV/checkpoint logs, while W&B should be treated as run-level metadata unless a row states otherwise. Stability Index, Peak-to-Tail Drift, and Rolling Variance are symptom-level reward-trajectory proxies; they are not measurements of per-token KL. This limits policy-drift claims, but it is separate from the ZVF/GU claim, which is computed directly from reward equality within sampled groups.

6.2 Methodological and Statistical Scope

A GRPO-style runner, not canonical GRPO. The main training runner preserves the critic-free, group-normalized-advantage structure needed for the ZVF/GU analysis, but it omits several elements often associated with canonical implementations: a PPO probability ratio and clipping term, a frozen reference-policy KL penalty, and a completion-only token mask in some runs. Therefore the paper's safest algorithmic claim is about a *group-relative critic-free RL loop under the reported stack*, not about all possible GRPO implementations.

Short horizons and limited replication. Most managed experiments use short horizons and single seeds, primarily because frontier-scale managed training is expensive and failure-prone. These runs are suitable for studying early learning-signal availability, collapse, saturation, and stack sensitivity. They are not sufficient for asymptotic convergence, state-of-the-art performance, long-horizon reward hacking, or robust algorithm ranking. Where multi-seed or held-out analyses exist, they are reported separately; otherwise numbers should be read descriptively, consistent with the replication cautions of Henderson et al. [32].

Held-out capability evidence is narrow. The clean held-out capability check is the separate GSM8K 200-example slice, which improves from 82.0% to 83.3% and is not statistically significant ($p = 0.26$). Consequently, the paper does not claim a significant held-out math improvement. Training rewards support statements about optimization dynamics inside a particular reward environment; they do not establish general reasoning, code, or tool-use capability.

Public benchmark and contamination risk. GSM8K, MATH-style prompts, and HumanEval are public benchmarks, and we cannot rule out that base models saw related examples during pretraining or instruction tuning. The held-out split is held out from *our fine-tuning loop*, not from the upstream training histories of the base models. Absolute performance therefore should not be interpreted as novelty on unseen tasks; the defensible comparison is relative to each run's initialization and logged reward environment.

Missing competitive baselines. The study does not exhaust strong alternatives such as SFT-only warm-starts, best-of-N sampling, rejection sampling, DAPO/Dr. GRPO-style corrections, or longer-budget PPO/GRPO variants under fully matched infrastructure. This is a limitation for performance claims. It is less damaging to the paper's main diagnostic contribution, because ZVF/GU ask whether the sampled reward groups contain contrast before any particular optimizer can exploit it.

ZVF/GU are triage diagnostics, not universal predictors. The empirical ZVF correlations are confounded by task type, and the fixed first-five-step rule is validated on a small de-duplicated set with only two positive collapse cases. We therefore do not claim that early ZVF alone ranks models or predicts final reward across arbitrary tasks. The stronger and more portable claim is mechanistic: under a group-relative objective with binary rewards, groups with identical rewards produce no reward-contrast signal, while mixed groups do. Monitoring ZVF/GU exposes whether the reward environment is in cold-start collapse, usable contrast, or saturation.

6.3 Reward-Proxy and Length-Bias Risks

Proxy rewards. Several environments intentionally measure proxies. The Tinker API tool-use rows score schema/function-call form rather than executed tool success; the HumanEval-style sandbox removes Python built-ins such as `len`, `range`, and `sum`; and the MATH reward mixes final-answer matching with `\boxed{}` formatting. The MATH-500 and HumanEval prompt pools used in the training loop are reward-environment probes rather than held-out generalization tests. These rows are useful for studying whether the rewarder supplies contrast, but they should not be upgraded into broad code, math, or tool-use claims.

Sparse rewards and tool-use failures. The Tinker API tool-use rows for Qwen3-32B and Llama-3.1-8B-Instruct score 0% under the strict schema reward. We interpret this as evidence that the sampled groups contained no usable positive reward under that verifier and horizon, not as evidence that the base models are incapable of tool use. SFT-warmed or curriculum-based tool-use rows in the broader artifact should be read as separate proxy-reward experiments with different initial conditions.

Length bias and reward hacking are not fully measured. Prior work shows that GRPO-like objectives can be sensitive to length bias and verbosity-trap dynamics [69, 132]. Our telemetry flags peak-to-tail reward regressions and verbosity-trap-like trajectories, but we do not uniformly log response length, direct KL, or long-horizon behavior across all runs. Thus the current data can show reward instability and motivate length-normalized objectives; it cannot prove that length alone caused the observed regressions. Dr. GRPO, length penalties, and sequence-normalized objectives remain untested mitigations in this artifact.

Net effect on the paper's claims. These limitations remove broad performance claims and universal algorithm rankings. What remains is deliberately narrower: ZVF/GU give a practical way to ask whether a verifier-style reward is producing the within-group contrast that a critic-free group-relative update needs. The paper's contribution is strongest when read as a diagnostic and reporting framework for RL post-training stacks, not as evidence that this runner generally improves model capability.

6.4 Broader Impact

Alignment and Misuse. GRPO and related RLHF methods are dual-use technologies. On the positive side, they are the primary mechanism by which frontier models are aligned to human preferences, and our audit artifact lowers the barrier to studying these methods rigorously. On the negative side, the same reward-maximization machinery can be directed at adversarial objectives—optimizing for reward-hack proxies, generating persuasive misinformation, or circumventing safety classifiers. We do not provide new attack capabilities, but improvements in RL post-training efficiency reduce the compute budget required for such misuse. We encourage the community to develop robust reward modeling and out-of-distribution detection in parallel with efficiency advances.

Democratizing RL-for-LLM Research. A primary motivation for TINKERRL AUDIT is to make rigorous RL post-training experiments accessible to researchers without large-scale compute. By publishing standardized tasks, Docker containers, and cost-annotated run logs, we aim to reduce the advantage that well-resourced labs hold in this space. Our total experimental expenditure was modest: approximately $120 in Tinker API credits (including unsuccessful or interrupted attempts), and roughly $95 in Modal compute for the six held-out evaluation jobs (three of which timed out). Open-source backend experiments were conducted on a single A100 node donated by our host institution's research group. We include these cost breakdowns explicitly so that other groups can budget accordingly.

Training Data and Privacy. All training data is drawn from publicly released datasets: GSM8K [19] for mathematical reasoning, HumanEval [15] for code generation, and a synthetic tool-use corpus generated without human subjects. No private data, personally identifiable information, or licensed proprietary content is used. The audit corpus does not involve human participants in any capacity, and no IRB review was required.

No Direct Dual-Use Concern. Our contribution is a *diagnostic audit framework*, not a new model or capability. We do not release any model trained on sensitive, harmful, or restricted content.

Evaluated checkpoints derive from publicly available base models under their respective licenses: Qwen model families under Apache-2.0; Llama-3.x model families under Meta's Llama Community License; DeepSeek-V3.1 and Nemotron under their respective community/commercial terms. We do not claim all bases are Apache/MIT-equivalent. We do not introduce a new hazardous dataset or attack method, but improved RL post-training workflows are dual-use in the general sense that any optimization advance can lower cost for misuse; we discuss this as a qualitative residual risk rather than claiming no direct risk.

Reproducibility Statement. Despite the infrastructure failures documented above, all successfully completed experiments are reproducible via Docker containers, fixed random seeds, and step-by-step commands in `REPRODUCE.md`. Corrected logging and KL-tracking code is included in the release. We have archived all local CSV training logs so that the step-level reward trajectories excluded from W&B are nonetheless available to other researchers.

7 Discussion

7.1 Why Might Stack-Conditioned Algorithm Effects Vary?

The sharpest observation in our audit corpus is not a stable algorithm ranking, but the fact that such a ranking is not identifiable from the available artifacts. The Qwen PPO/GRPO row changes interpretation depending on which result summary is treated as canonical, while PPO is substantially higher in the recorded Llama-3.1-8B setting (Mann-Whitney $r = 0.94$, $p < 10^{-10}$ on per-step rewards). However, these rows are single-seed and backend-confounded, and the Mann-Whitney p-value assumes independent observations while per-step rewards within a single trajectory are autocorrelated. We present three mechanistic hypotheses that future work can test with proper multi-seed replication.

Hypothesis 1: Reward landscape geometry. GRPO replaces the value-function baseline of PPO with a within-group reward mean, computing advantages purely from peer comparisons. This estimator is low-variance when the per-prompt reward surface is smooth enough that sampled completions span a reliable gradient direction — a condition more likely to hold when the model already has strong prior task performance. Qwen3-8B enters GSM8K training at 89.8% zero-shot accuracy, providing a reward landscape that is largely flat with narrow high-reward ridges; GRPO's group-relative contrast effectively navigates these ridges without a value function approximation error. Llama-3.1-8B enters at a lower zero-shot baseline, presenting a rougher landscape where the value function's global smoothing is beneficial. This conjecture predicts that, under similar verifier-reward and sampling conditions, group-relative methods may be more competitive when the model already samples some correct completions, whereas value-function methods may help when the initial reward landscape is rougher. Our data are too small and confounded to support a prescriptive GRPO-vs-PPO rule.

Hypothesis 2: Instruction-tuning quality modulates advantage noise. Both Qwen3-8B and Llama-3.1-8B are instruction-tuned, but they differ in instruction-following fidelity and output format consistency. Inconsistent formatting inflates the within-group reward variance for reasons unrelated to mathematical reasoning quality, injecting noise into GRPO's advantage signal. PPO's value function, trained jointly on the task, can absorb some of this format-related variance and focus the policy gradient on reasoning-relevant features. This hypothesis is consistent with our MoE finding: the base (non-instruct) Qwen3-30B-A3B achieves only 50% peak GRPO reward, while the instruction-tuned variant reaches 100% — the initialization quality effect is substantial and independent of architecture.

Hypothesis 3: Reference-policy proximity. GRPO's objective is reference-free by design [110], but the de-facto reference is the initialization checkpoint. Models that are "closer" to the target distribution at initialization (i.e., have been trained on reasoning-style data) exhibit lower ZVF and smaller effective KL excursions during training. Qwen3's training lineage includes substantial mathematical reasoning data; Llama-3.1's instruction tuning is more general-purpose. The lower ZVF we observe for Qwen3-family experiments (8.5% mean for GSM8K) relative to the instability patterns in Nemotron-120B (SI $= 1.180$) supports this proximity argument. Direct testing requires

gradient-level analysis of the kind described in Liu et al. [70], and we release our checkpoints expressly to enable such analysis.

Causal test of the ZVF/GU hypothesis. To test whether ZVF/GU is merely observational or actually causal, we propose a controlled experiment that stratifies GSM8K prompts by measured base-policy hit rate and trains matched GRPO runs that differ only in prompt-pool reward diversity. Three arms would be run with Qwen3-8B and identical GRPO config: (1) dead prompts (base hit rate $< 25\%$), (2) mixed prompts (base hit rate 25%–75%), and (3) saturated prompts (base hit rate $> 75\%$). The primary endpoint is first-5-step gradient utilization and reward slope, not held-out accuracy: the thesis is not that GRPO improves GSM8K, but that GRPO has a learning signal only under reward diversity. The expected pattern is: dead and saturated arms show ZVF ≈ 1.0 and flat reward curves; the mixed arm shows lower ZVF and positive reward slope. Running this experiment would turn the current observational ZVF/GU diagnostic into a direct causal test.

7.2 What Stack Sensitivity Means for the Field

The classical RL-library rows (SB3, CleanRL, Tianshou) are useful engineering sanity checks, but they are not evidence for a framework ranking in LLM post-training. Their near-random arithmetic performance primarily shows that a text-generation policy cannot be treated as a standard continuous-control MDP without explicitly specifying tokenization, autoregressive rollout, loss masking, reward parsing, and action construction. This is stack mis-specification evidence, not evidence that PPO-the-algorithm fails.

The practical implication is narrower and stronger: published comparisons such as "PPO versus GRPO" are underidentified unless the realized treatment is specified. A defensible comparison must state the model checkpoint and tokenizer, prompt template, reward parser, sampler and grouping process, loss mask, KL/reference handling, optimizer, LoRA targets, precision/backend, and evaluator. Without those details, an algorithm label is not enough to tell whether two rows are comparable.

7.3 Connections to Concurrent Work

AERO and ZVF as a training diagnostic. Zhang et al. [149] motivate AERO by the prevalence of zero-advantage dead zones in GRPO training and propose a Bayesian posterior over prompt difficulty to pre-screen prompts. Our ZVF measurement ($r = -0.769$, $p < 0.001$ with final performance) provides an empirical calibration point for the phenomenon AERO targets, while the lagged analysis keeps the claim scoped: ZVF is useful for triage, not as a standalone performance predictor. Importantly, our data reveal that ZVF is *task-driven, not model-scale-driven*: tool-use experiments saturate at ZVF $= 100\%$ regardless of model size, while GSM8K experiments average ZVF $= 8.5\%$ across the same model range. This suggests that AERO's compute savings will be largest when the task is poorly matched to the model's prior capabilities — precisely the regime where standard GRPO training is most wasteful. Our ZVF measurements can serve as a calibration dataset for AERO's Beta prior.

Descriptive saturation fits and their limits. Nimmaturi et al. [84] derive a three-phase exponential saturation law for GRPO, finding that 80% of training steps contribute marginally and proposing early stopping. We observe a compatible three-phase pattern in 73% of LLM fine-tuning experiments, while treating it as descriptive because the traces are short. However, our data reveal a critical boundary condition: the saturation framework *fails for unstable training runs*. Nemotron-120B's trajectory (87.5% peak $\rightarrow$ 16.2% last-10) is non-monotonic and cannot be fit by the exponential saturation model — it is better characterized as a reward excursion followed by policy collapse, a regime the scaling law does not model. We recommend that practitioners verify monotonicity before applying early stopping rules derived from the saturation model.

"It Takes Two" and the 2-GRPO/DPO equivalence. Wu et al. [135] prove that GRPO is algebraically equivalent to an implicit contrastive objective (DPO), with group size G affecting only the Monte Carlo variance of that objective. Their headline finding: **2-GRPO** ($G = 2$) retains 98.1% of 16-GRPO performance while requiring 12.5% of rollouts and 21% of training time.

Liu et al. [70] show that the DPO objective can be viewed as a contrastive objective over a subnetwork of parameters, and that the subnetwork's size determines the KL per parameter. Our finite-group

analysis recovers the same expected-ZVF expression: $\mathrm{GU}(p, 2) = 2p(1-p)$, which is exactly the Bernoulli variance of the per-prompt accuracy p and reproduces the DPO preference weighting. For our DeepSeek-V3.1 run ($p \approx 0.85$, $G = 8$), 27% of steps were zero-gradient; switching to $G = 2$ would preserve all informative contrastive signal while reducing rollout overhead by 75%. However, the DPO equivalence also predicts that very high-accuracy models ($p > 0.9$) will have low gradient utilization regardless of G, suggesting that the relevant intervention at high accuracy is not group-size reduction but prompt re-sampling from harder sub-distributions.

Target Policy Optimization and sparse-reward regimes. The 0% tool-use reward we observe for both Qwen3-32B and Llama-3.1-8B exemplifies the sparse-reward failure mode that Kaddour et al. [48] specifically designs against: without any positive reward signal in a group, the advantage is uniformly zero and no gradient flows. TPO's target-distribution construction ($q \propto p_{\text{old}} \exp(u)$) provides a training signal even when all rollout rewards are zero, by fitting to the *implied* contrastive distribution rather than observed rewards. Our tool-use results provide an empirical motivation for this class of approaches: the task is not unsolvable in principle (Qwen2-0.5B achieves functional tool calls with SFT), but it is unsolvable with binary GRPO rewards at 30-step horizons. We recommend TPO-style objectives as the first algorithm to try when ZVF $= 1$ persists past the first five training steps.

7.4 Limitations of Proxy Metrics and the Path to Direct KL Measurement

Our reward-instability analysis relies on three reward-trajectory proxies (SI, PTD, Rolling Variance) that are associated with late reward in this artifact ($r = -0.517$, $p = 0.005$ for PTD, uncorrected) but are not the same as direct KL divergence measurements. The proxies capture *observable symptoms* of reward instability — reward instability and peak-to-tail degradation — but cannot distinguish between two importantly different causes: (a) the policy has genuinely drifted far from the reference in token-distribution space, or (b) the reward function is inherently noisy and the policy is stable but reward variance is high. Nemotron-120B's collapse is almost certainly case (a), given that its reward decline is monotonic and persistent; but for models with moderate PTD (0.1–0.3), the two causes are indistinguishable from reward trajectories alone.

Corrected KL tracking code is released in `src/grpo_trainer.py` and should be validated on at least the two Modal runs (Qwen3-8B and Llama-3.1-8B) where GPU access is reproducible. Additionally, the per-token divergence framework of Liu et al. [70] — which measures which parameters actually update during RL fine-tuning — could reveal whether the small subnetwork they identify has higher KL per parameter, potentially explaining why moderate-sized models sometimes exhibit reward collapse consistent with drift (rather than direct evidence of catastrophic drift, which would require direct KL or policy-distance telemetry we do not have) despite updating only 5–30% of weights. Connecting our behavioral proxy metrics to this parameter-level analysis is a natural and important next step.

8 Conclusion

This study does not show that GRPO generally improves reasoning, and we are careful not to claim that it does. The training runner is a group-relative policy-gradient variant inspired by GRPO rather than a faithful implementation of the canonical objective (missing PPO clip, reference-KL, and completion-only mask; a P1-A diagnostic measures ∼76% of the full-sequence loss magnitude to be prompt-token gradient), so algorithm-level conclusions apply to this runner and not to GRPO-the-algorithm. Three of our reward environments (HumanEval, tool-use, and MATH) measure sandbox-restricted execution, JSON schema compliance, and boxed-answer formatting respectively, not end-to-end capability. The MATH-500 and HumanEval prompt pools in the training loop are loaded from the `test` splits and function as reward-environment probes, not as generalization tests. The only capability signal evaluated on a genuinely held-out slice is GSM8K, where the lift from 82.0% to 83.3% is not significant ($p = 0.26$). The defensible conclusion is that this critic-free group-relative RL loop behaves as a conditional reward-contrast amplifier: it can refine behavior when the current policy samples both successes and failures under a verifier that rewards them, and it cannot reliably bootstrap competence when sampled groups are flat or the verifier rejects legitimate completions.

The paper's main contribution is therefore diagnostic. ZVF and GU should be logged early and interpreted jointly with reward level. High ZVF with low reward is a cold-start alarm; high ZVF with high reward is saturation; low ZVF means rollout compute is still producing contrast. In our validation set, the first-five-step rule catches the two collapsed runs without false positives, but the low number of failures and weak continuous-ranking result mean ZVF/GU should be treated as triage diagnostics, not performance predictors.

The mechanism behind that diagnostic is mixed-group probability:

$$P(\text{usable}) \approx \frac{1}{N} \sum_{x} \left[1 - (1 - p_x)^G - p_x^G\right].$$

This formula is the cleanest way to state the boundary condition. The expected-one-success rule $p_0 > 1/G$ is only a heuristic. Future work should test the three direct interventions implied by the equation: raise p_0 with SFT or easier curricula, raise G where rollout cost allows, and densify the reward so equivalent partial progress is not collapsed into all-zero groups.

Finally, algorithm labels should be reported as stack-conditioned results. The Qwen PPO/GRPO row is artifact-sensitive, and the Llama row favors PPO; neither supports a universal PPO/GRPO ordering. A publishable comparison needs the full treatment definition: model family, backend, reward parser, sampler, optimizer, KL/reference handling, LoRA configuration, checkpoint selection, and held-out evaluation. Without those controls, "PPO versus GRPO" is not an interpretable scientific claim.

9 Ethics, Limitations, and Broader Impact

This statement is written to satisfy the NeurIPS Code of Ethics and Broader Impact requirements. It complements the shorter Limitations and Broader Impact subsections embedded in Section 6 by consolidating in one place a full dual-use analysis, itemised compute accounting, carbon footprint estimation, data provenance disclosures, a candid acknowledgment of the closed-source training backend on which a fraction of our headline numbers depends, and a list of methodological limits we are aware of but did not have resources to close within the submission window.

9.1 Dual-Use Analysis: Misuse Risks of Reasoning RL

Threat model. TINKERRL AUDIT releases (i) training scripts that apply GRPO to the GSM8K [19] mathematical reasoning benchmark and to the Salesforce xLAM-Function-Calling-60k [65] tool-use corpus, (ii) a set of LoRA adapters published on the HuggingFace Hub (`anonymous/tinker-rl-bench-*`), and (iii) diagnostic code for Zero-Variance Fraction and reward-instability analysis. We analyse four concrete misuse pathways these artefacts could, in principle, enable, and we describe the existing guardrails and the residual risk we judge acceptable for publication.

M1. Weaponisation of mathematical reasoning. The most capability-relevant artefact we release is code for running GRPO-style verifier-reward experiments on grade-school arithmetic and multi-step prompt suites. In our clean held-out GSM8K control, the measured gain is small and not statistically significant, so we do not claim the code improves general reasoning capability. A malicious operator could attempt to extend this pipeline to tasks of greater dual-use concern, e.g., chemistry olympiad problems, cryptographic key recovery, or financial fraud. We judge the *marginal* uplift provided by our release to be small for three reasons. First, GRPO at our training scale does not create new capabilities; the base-model control for GSM8K (Section **??**, 82.0%) shows that the bulk of held-out accuracy is attributable to Qwen3-8B's pretraining. Our full GRPO run only adds $+1.3$ percentage points (p=0.26, not statistically significant). Second, the public literature already contains GRPO implementations [110, 90, 111, 125] that are at least as capable. Third, any reward-harness improvement is bounded by the rewarder: GSM8K rewards use exact-match against a human-written gold answer and do not transfer to the domains listed above without a new validated reward signal, which itself requires expertise and labelled data.

M2. Reward hacking and the long-term alignment risk. GRPO, like all policy-gradient RL from a proxy reward, is vulnerable to reward hacking: the model learns to exploit idiosyncrasies of the reward function rather than solve the intended task [116, 52]. Our reward-instability audit

(Section 6.3) flags verbosity-trap-like regressions in 4/11 GRPO trajectories, but treats them as proxy-reward failures rather than proof that response length alone caused the regression. Users who apply our scripts to under-specified reward functions in production settings (e.g., a custom "helpfulness" classifier) may observe models that appear to improve on held-out metrics while silently optimising against unintended proxies. We include the Zero-Variance Fraction and reward-stability diagnostics precisely so that downstream users can detect such drift.

M3. Circumventing safety training via distillation or tool-use. The tool-use track trains LoRA adapters that teach a base model to emit schema-valid JSON function calls [65]. A motivated adversary could in principle use the same pipeline to teach a base model to emit jailbreak prompts as "tool calls" or to invoke an adversarial external tool. We mitigate this by (a) releasing only adapters, not merged weights, so that the safety-trained instruct checkpoint must be applied separately and any instruct-layer guardrails remain active; (b) documenting in model cards that the tool-use adapters are trained on a narrow schema (five synthetic tools) and have not been safety-evaluated for open-ended function calling; and (c) not releasing any function-calling harness that would actually execute emitted calls.

M4. Compute-efficiency externalities. Our work argues that critic-free RL with careful group-size selection ($G{=}32$) can be run on modest budgets. Improved training efficiency is itself dual-use: it lowers the cost of running adversarial fine-tuning at scale. We judge this externality to be modest relative to the already-low cost of fine-tuning a 0.6B–8B model with commodity LoRA recipes [40], but we flag it explicitly.

Residual risk. After weighing M1–M4 we judge that the reproducibility, calibration, and pedagogical benefits of releasing TINKERRL AUDIT outweigh the residual misuse risk. We adopt the standard mitigation of releasing adapters (not merged weights), documenting intended use and out-of-scope use in model cards, and publishing alongside the diagnostics a researcher would need to detect reward hacking.

9.2 Compute Cost Accounting

Table 17 itemises the financial cost of every platform used in the project. Costs are real spend for the submitting authors; no in-kind credits are omitted. Dollar figures are in USD.

Table 17: Itemised compute spend across platforms. Totals include successful, interrupted, and unsuccessful attempts to reflect the actual cost of the audit [32].

Platform	Use	Runs	Cost (USD)
Tinker SDK v0.16.1	GRPO on 4B/8B (original suite)	25	\$40–55
Tinker SDK v0.16.1	10x Structural Ceiling ablation	32	\$65
Tinker SDK v0.16.1	Larger-model support suite (frontier/MoE)	13	\$25–30
Modal H100	PPO baselines (Qwen3-8B, Llama-3.1-8B)	2	\$12–18
Modal H100	KL-tracking run (failed)	1	\$2–4
Google Colab Pro (T4)	0.5B–3B QLoRA SFT+GRPO	—	\$10/person
NVIDIA L4 (TRL baseline)	Qwen2.5-0.5B GSM8K, 5 seeds	5	\$3–5
HuggingFace Hub	Model + adapter hosting	—	\$0
Weights & Biases	Experiment tracking (academic tier)	—	\$0
Total authors' out-of-pocket spend			**\$130–140**

Hidden costs. Beyond direct platform spend, the project consumed human labour (approximately six student-FTE-months, unpaid) and infrastructure generously subsidised by Anonymous Institution's LLM Research Group (shared access to one A100 node used for TRL baselines and pre-submission sanity checks). No author received commercial compensation from Thinking Machines, Modal, or any other vendor mentioned.

Failed-run accounting. Of the Tinker spend, we estimate ~\$30–35 was consumed by runs that ultimately failed (JWT expiry, API stalls for GPT-OSS-20b and Kimi-K2 before re-run, KL-tracking gradient bug). We disclose this because reviewers often ask whether reported total spend reflects only

clean, successful runs; it does not. Excluding failed spend would understate the true resource cost of reproducing our work by roughly 25%.

9.3 Carbon Footprint Estimation

Table 18 computes an energy and CO_2-equivalent estimate for each platform. We follow the methodology of Patterson et al. [95] and Strubell et al. [119]: for each GPU-hour we multiply device TDP by wall-clock GPU-hours and the data-centre Power Usage Effectiveness (PUE), then multiply by the grid carbon intensity in the region where the hardware is believed to be located.

Assumed constants.

- NVIDIA H100 SXM5 TDP: 700 W.
- NVIDIA A100 80GB TDP: 400 W.
- NVIDIA L4 TDP: 72 W.
- NVIDIA T4 TDP: 70 W.
- Data-centre PUE: 1.1 (conservative; typical hyperscale PUE is 1.10–1.15 [27, 4]).
- US average grid intensity (EPA eGRID 2023): 0.367 kg CO_2-eq / kWh [124].
- Indian average grid intensity (CEA 2023): 0.716 kg CO_2-eq / kWh [14].
- We use the US average for Modal H100 (US-East region as configured) and Tinker (location undisclosed; assumed US), and the Indian average for Colab Pro T4 used by authors based in Bengaluru (Google Cloud asia-south1).

GPU-hour estimates. Tinker GPU-hour counts are not disclosed by the platform. We estimate them from wall-clock run duration and assume a single H100-class accelerator per job, which is consistent with Tinker's published architecture for the model sizes we trained. Modal GPU-hours are measured directly from Modal's billing dashboard.

Table 18: Energy and carbon footprint estimates. Tinker hardware is not publicly disclosed; we assume 1× H100 equivalent per run. Failed / stalled runs are included. Totals rounded to one significant figure.

Platform	GPU-h	TDP (W)	PUE	Energy (kWh)	CO_2 (kg)
Tinker (assumed H100)	~950	700	1.1	~732	~269
Modal H100 (US-East)	~18	700	1.1	~14	~5.1
Institutional A100 (in-kind)	~60	400	1.1	~26	~19
Colab Pro T4 (asia-south1)	~40	70	1.2	~3.4	~2.4
NVIDIA L4 (TRL baseline)	~10	72	1.1	~0.8	~0.3
Project total (best estimate)				~776	~296

Interpretation and uncertainty. Our best estimate of ~296 kg CO_2-eq for the entire project is comparable to a single round-trip economy flight Bengaluru–Delhi (~300 kg). The dominant uncertainty is the Tinker GPU-hour count: because Tinker does not expose hardware telemetry, a factor-of-two error in GPU-hours or TDP is plausible. Under a pessimistic assumption (2× GPU-hours, H100 running near 100% TDP continuously), the Tinker contribution could be as high as ~540 kg CO_2-eq. Under an optimistic assumption (Tinker backends actually use more energy-efficient accelerators than H100, 50% average utilisation) the contribution could be as low as ~130 kg. We report the central estimate in the main table and caution readers that it should not be interpreted as precise. *All* numbers should be regarded as upper bounds on the carbon cost of reproducing our work, because subsequent users can skip unsuccessful attempts and optional ablation sweeps.

Offset and mitigation. We did not purchase carbon offsets. Instead, we have (a) released all trained checkpoints on HuggingFace Hub so that downstream users do not need to re-run our experiments; (b) released step-level CSV logs so that learning curves can be inspected without re-training; and (c) documented in Section 9.2 which runs were wasteful, so that replicators can skip them. We regard reproducibility-by-artefact as a more durable mitigation than offsets.

9.4 Data Provenance

TINKERRL AUDIT uses only publicly released research datasets. No private data, personally identifiable information, or licensed proprietary content is used. No human annotators were employed during this work. We document each dataset below.

GSM8K [19]. 8,500 grade-school math word problems (7,473 train / 1,319 test) authored by human writers contracted by OpenAI. Released under the **MIT License** via `https://github.com/openai/grade-school-math`. We use the standard train/test splits without modification. The canonical citation is Cobbe et al. [19]. Known limitations of GSM8K include gender-neutral but culturally US-centric word problems (names, currencies, sports) and a moderate rate of human-labelling errors estimated at ~2% by [151]. Our held-out evaluation respects the test/train split.

Salesforce xLAM-Function-Calling-60k [65]. 60,000 function-calling examples generated by Salesforce Research as training data for the xLAM agentic model family. Released under the **CC BY 4.0** license via `https://huggingface.co/datasets/Salesforce/xlam-function-calling-60k`. We use the public release as-is; specifically, we use the first 35 prompts $\times$ 10 rollouts in the 10x Structural Ceiling tool-use track and the full 60k split for the xlam-60k real-data run in Section **??**. Known limitations: xLAM schemas are synthetic and skew toward cleanly-typed arguments; the dataset under-represents ambiguous, error-handling, and multi-turn tool-calling. Attribution to Salesforce is required by the license and is provided in Section 9.6 and in the xLAM-derived model cards.

HumanEval [15]. 164 Python programming problems with unit tests, released by OpenAI under the **MIT License**. Used unmodified for pass@k evaluation. Known limitations: small scale, English-language docstrings, single-language coverage; documented contamination concerns against frontier pretraining corpora [104].

NuminaMath [58]. Used by an anonymous collaborator for the multi-stage GSM8K+NuminaMath pipeline. Released under **Apache 2.0**. We use a downstream checkpoint reported by that collaborator; our repository contains only his scripts and the Apache-licensed derivative weights.

Open-Platypus [55]. 3,000 SFT examples used by an anonymous collaborator for Qwen3-8B code generation warm-up. Open-Platypus is a curated subset released under **CC BY-NC 4.0**; the non-commercial restriction is respected in our release, which is academic-use only.

Synthetic tool-use corpus (authored). We additionally generated a small five-tool synthetic corpus (calculator, web-search stub, calendar, file-read, email-send) used for Section 4.1's saturation analysis. The corpus is authored by the paper's authors from publicly documented APIs, contains no third-party content, and is released under the **MIT License** in our repository under `data/synthetic_tools/`. Generation prompts are included for auditability. No user data, scraped web content, or proprietary API traces are present.

No web scraping, no human subjects. We did not scrape any website. We did not run any study with human participants. No IRB review was therefore required. No data that could reasonably be considered to contain PII, copyrighted content, or sensitive personal attributes was used at any stage of training, evaluation, or reward modelling.

9.5 Closed-Source Tinker Acknowledgment

A central tension in TINKERRL AUDIT is that a substantial fraction of our headline numbers come from a closed-source commercial platform while our paper simultaneously advocates for reproducibility and platform independence. We do not resolve this tension by hiding it; we describe it precisely.

What Tinker is and is not. Tinker [122] is a managed LLM fine-tuning and inference service provided by Thinking Machines, Inc. It exposes a Python SDK (`tinker==0.16.1`) that accepts custom loss functions (`forward_backward_custom`), a limited set of optimisers, and standard LoRA

hyperparameters. It does not expose (i) the exact server-side GRPO loss implementation, (ii) reward normalisation or baseline subtraction scheme, (iii) minibatch construction or gradient-accumulation strategy, (iv) hardware configuration (GPU type, inter-node bandwidth), or (v) system-level telemetry (energy, throughput, queueing).

What this means for our claims. Tinker results in this paper measure the *platform's* implementation of GRPO, not an abstract specification of the algorithm. A reader who asks "why do nominally similar managed and TRL GRPO runs produce different reward trajectories" cannot fully answer that question from our data: visible hyperparameters are matched in some probes, but hidden sampler, loss-mask, LoRA, precision, rollout, checkpoint, and evaluator details remain part of the treatment. We therefore treat managed and TRL rows as stack-conditioned evidence, not as a causal framework ranking. Dry-run verl/OpenRLHF launcher rows are engineering-readiness artifacts only and are not used for empirical framework conclusions.

Reproducibility commitments we can make.

1. All Tinker experiment scripts, configuration files, JSON step logs, and per-run W&B projects are archived in the repository. Researchers with Tinker access can attempt replication with our exact configurations.
2. Every figure and every summary statistic derived solely from Tinker data is marked with "†" and a footnote stating that independent replication requires Tinker API access.
3. Primary statistical claims distinguish real runs from dry-run launchers and separate online reward from held-out capability. Managed-only results are reported descriptively unless accompanied by a matched held-out control.
4. We commit to re-running any Tinker-only experiment on an open backend if an equivalent open-source service becomes available, and to issuing a revised version of this paper if a stack variable changes the interpretation of any headline result.

Key rotation and credential hygiene. Our Tinker API key (prefix `tml-...`) is held in a repository `.env.example` placeholder only. The real key is stored in the authors' password manager and rotated whenever a failure mode suggests possible exfiltration. No Tinker key is committed to git history in either the primary repository (`anonymous-org/tinker-rl-lab`) or the mirror (`anonymous-mirror/tinker-rl-lab`).

9.6 Known Methodological Limits

In addition to the infrastructure failures and platform-specific caveats enumerated in Section 6, we flag the following methodological limits.

Short training horizons. All Tinker GRPO runs were capped at 30–50 gradient steps. This is a deliberate cost-control choice and means our Tinker numbers are *early-training snapshots*. We cannot rule out that longer training would change the qualitative picture (e.g., the 82%→83.3% held-out gain might widen, or might collapse to reward hacking).

Single-seed Tinker experiments. Each Tinker configuration was run once, not 3–5 times as best practice prescribes [32]. Tinker results therefore lack variance estimates and do not support significance testing; we report them descriptively. Only the 5-seed TRL baseline and the 5-seed held-out GSM8K evaluation carry proper confidence intervals.

Train-set reward as primary Tinker metric. Tinker runs primarily report reward on training prompts. Only the GSM8K track was followed up with a separate 200-example held-out evaluation per seed. Other Tinker tracks (tool-use, xLAM) remain train-set-only and we cannot distinguish memorisation from generalisation for those settings.

LoRA only; no full fine-tuning. All experiments use LoRA [40] with ranks 8–64. We have *not* tested whether full fine-tuning would re-order the library/algorithm comparisons. The LoRA constraint is practical (cost) but it means our claims about PPO vs. GRPO and TRL vs. Tinker are LoRA-conditional.

Benchmark coverage is narrow. We evaluate four domains: GSM8K, MATH-500 (exploratory), HumanEval (subset), and synthetic/xLAM tool-use. We do not evaluate on MT-Bench, ArenaHard, safety benchmarks (HarmBench, ToxicChat), or truthfulness benchmarks (TruthfulQA). Any claim about "reasoning" in the paper is strictly a claim about the covered benchmarks.

No human preference data. We use verifiable rewards only (exact-match for math, unit-test for code, schema-validity for tool-use). We do not study RLHF with human preference data; findings should not be extrapolated to reward-model–based alignment without further work.

Closed-source implementation opacity (reiterated). Comparisons between Tinker GRPO and TRL GRPO are confounded by the closed-source nature of Tinker's implementation. See Section 9.5 for detail.

Cross-platform hardware confounding. Tinker, Modal, Colab, and the Institutional A100 node use different accelerators, different memory hierarchies, and different inference/training stacks (Tinker proprietary, Modal vLLM, Colab PyTorch-native, TRL+Accelerate). Observed differences in reward dynamics are not purely algorithmic.

Carbon estimates are approximate. As discussed in Section 9.3, Tinker GPU-hour and TDP are inferred, not measured. Grid intensity is region-average, not marginal. Numbers in Table 18 should be treated as order-of-magnitude.

Exploratory, not confirmatory. This is an exploratory case study, not a pre-registered confirmatory experiment. We did not pre-register primary hypotheses. All p-values are un-corrected for the number of comparisons actually made across the paper; the Bonferroni-surviving comparisons in Section **??** are so flagged, but most of our secondary analyses are descriptive.

Geographic and demographic limits. All authors are based at a single institution in Bengaluru, India. Our choice of benchmarks, prompt phrasings, and evaluation design reflects that context. Independent replication by groups in other regions, on non-English tasks, and with non-Qwen / non-Llama families is an open question.

References

[1] ACM. Artifact review and badging – version 1.1, 2020. URL `https://www.acm.org/publications/policies/artifact-review-and-badging-current`. Accessed: 2026-04-13.

[2] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville, and Marc Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, volume 34, pages 29304–29320, 2021.

[3] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2402.14740. arXiv:2402.14740.

[4] Amazon Web Services. AWS sustainability: Data center efficiency and PUE. `https://sustainability.aboutamazon.com/`, 2023.

[5] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint*, 2021. doi: 10.48550/arXiv.2108.07732. arXiv:2108.07732; MBPP.

[6] Mohammad Gheshlaghi Azar, Zhaohan Daniel Guo, Bilal Piot, Rémi Munos, Mark Rowland, Michal Valko, and Daniele Calandriello. A general theoretical paradigm to understand learning from human preferences. 2024. IPO.

[7] Sanghwan Bae, Jiwoo Hong, Min Young Lee, Hanbyul Kim, Jeongyeon Nam, and Donghyun Kwak. Online difficulty filtering for reasoning oriented reinforcement learning. In *Proceedings*

of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), pages 700–719, Rabat, Morocco, March 2026. Association for Computational Linguistics. ISBN 979-8-89176-380-7. doi: 10.18653/v1/2026.eacl-long.30. URL https://aclanthology.org/2026.eacl-long.30/.

[8] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint*, 2022. doi: 10.48550/arXiv.2204.05862. arXiv:2204.05862.

[9] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint*, 2022. doi: 10.48550/arXiv.2212.08073. arXiv:2212.08073.

[10] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1):289–300, 1995. doi: 10.1111/j.2517-6161.1995.tb02031.x.

[11] ByteDance Seed, Yu Yue, Yufeng Yuan, Qiying Yu, Xiaochen Zuo, Ruofei Zhu, Wenyuan Xu, Jiaze Chen, Chengyi Wang, Tiantian Fan, et al. VAPO: Efficient and reliable reinforcement learning for advanced reasoning tasks. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2504.05118. arXiv:2504.05118.

[12] Casimiro Carrino et al. Rethinking clipping: REINFORCE with group relative advantage matches GRPO. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2603.11882. arXiv:2603.11882; RGRA.

[13] Emanuele Cavenaghi, Gabriele Sottocornola, Fabio Stella, and Markus Zanker. A systematic study on reproducibility of reinforcement learning in recommendation systems. *ACM Transactions on Recommender Systems*, 1(3):1–30, 2023. doi: 10.1145/3596519.

[14] Central Electricity Authority, Government of India. CO_2 baseline database for the Indian power sector, user guide version 19.0. https://cea.nic.in/cdm-co2-baseline-database/, 2023.

[15] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[16] Wei Chen and Zhiyuan Li. Octopus v4: Graph of language models. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2404.19296. arXiv:2404.19296.

[17] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in Neural Information Processing Systems*, 30, 2017.

[18] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[19] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[20] Cédric Colas, Olivier Sigaud, and Pierre-Yves Oudeyer. A hitchhiker's guide to statistical comparisons of reinforcement learning algorithms. *arXiv preprint*, 2019. doi: 10.48550/arXiv.1904.06979. arXiv:1904.06979.

[21] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

[22] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, et al. DeepSeek-V3 technical report. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2412.19437. arXiv:2412.19437.

[23] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2501.12948. arXiv:2501.12948.

[24] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645 (8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.

[25] Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. KTO: Model alignment as prospect theoretic optimization. In *International Conference on Machine Learning*, 2024.

[26] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning*, 2023.

[27] Google. 2023 environmental report: Data center PUE. `https://sustainability.google/reports/`, 2023.

[28] Google DeepMind. Gemini 2.5 Pro: Advanced reasoning with Deep Think, 2025. URL `https://deepmind.google/technologies/gemini/`.

[29] Xinyu Guan, Li Lyna Zhang, Yifei Liu, Ning Shang, Youran Sun, Yi Zhu, Fan Yang, and Mao Yang. rStar-Math: Small LLMs can master math reasoning with self-evolved deep thinking. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2501.04519. arXiv:2501.04519.

[30] Ali Hatamizadeh, Greg Heinrich, Wenhan Han, Hongxu Yin, Jiaming Yang, Wonmin Byeon, Jan Kautz, and Pavlo Molchanov. iGRPO: Iterative group relative policy optimization for reasoning language models. In *Advances in Neural Information Processing Systems*, 2026. NeurIPS 2026.

[31] Alex Havrilla, Maksym Zhuravinskyi, Duy Phung, Aman Tiwari, Jonathan Tow, Stella Biderman, Quentin Anthony, and Louis Castricato. trlX: A framework for large scale reinforcement learning from human feedback. 2023.

[32] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. doi: 10.1609/aaai.v32i1.11694.

[33] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

[34] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks Track*, 2021.

[35] Jacob Hilton, Jie Tang, and John Schulman. Scaling laws for single-agent reinforcement learning. *arXiv preprint*, 2023. doi: 10.48550/arXiv.2301.13442. arXiv:2301.13442.

[36] Andreas Hochlehnert et al. A sober look at progress in language model reasoning. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2504.07086. arXiv:2504.07086.

[37] Matthew W. Hoffman, Bobak Shahriari, John Aslanides, Gabriel Barth-Maron, et al. Acme: A research framework for distributed reinforcement learning. In *arXiv preprint*, 2022. doi: 10.48550/arXiv.2006.00979. arXiv:2006.00979.

[38] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint*, 2022. doi: 10.48550/arXiv.2203.15556. arXiv:2203.15556.

[39] Jiwoo Hong, Noah Lee, and James Thorne. ORPO: Monolithic preference optimization without reference model. In *Conference on Empirical Methods in Natural Language Processing*, 2024.

[40] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL `https://openreview.net/forum?id=nZeVKeeFYf9`.

[41] Jian Hu, Xibin Wu, Zilin Zhu, Xianyu, Weixun Wang, Dehao Zhang, and Yu Cao. OpenRLHF: An easy-to-use, scalable and high-performance RLHF framework. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2405.11143. arXiv:2405.11143.

[42] Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, et al. MiniCPM: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2404.06395. arXiv:2404.06395.

[43] Shuhao Hu, Chenyu Wang, et al. OpenVLthinker: Multimodal reasoning with G^2RPO. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2603.04567. arXiv:2603.04567.

[44] Jin Huang, Harrie Oosterhuis, Bunyamin Cetinkaya, Thijs Rood, and Maarten de Rijke. State encoders in reinforcement learning for recommendation: A reproducibility study. In *Proceedings of the 45th International ACM SIGIR Conference*, pages 2738–2743. ACM, 2022. doi: 10.1145/3477495.3531716.

[45] Shiki Ichihara et al. MO-GRPO: Automatic variance-based reward reweighting for multi-objective alignment. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2510.17711. arXiv:2510.17711.

[46] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2503.09516. arXiv:2503.09516.

[47] Emma Jordan, Adam White, Bruno Castro da Silva, Martha White, and Philip S. Thomas. Position: Benchmarking is limited in reinforcement learning research. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2406.16241. arXiv:2406.16241.

[48] Jean Kaddour et al. Target policy optimization: Decoupled target construction for language model RL. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2601.12034. arXiv:2601.12034; TPO.

[49] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint*, 2020. doi: 10.48550/arXiv.2001.08361. arXiv:2001.08361.

[50] Kimi Team. Kimi K2: Open agentic intelligence. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2507.20534. arXiv:2507.20534.

[51] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1.5: Scaling reinforcement learning with LLMs. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2501.12599. arXiv:2501.12599.

[52] Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Jan Leike, and Shane Legg. Specification gaming: The flip side of AI ingenuity. DeepMind Safety Research blog, 2020. `https://deepmindsafetyresearch.medium.com/specification-gaming-the-flip-side-of-ai-ingenuity-c85bdb0deeb4`.

[53] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. doi: 10.1145/3600006.3613165.

[54] Thanh-Long V. Le, Myeongho Jeon, Kim Vu, Viet Dac Lai, and Eunho Yang. No prompt left behind: Exploiting zero-variance prompts in LLM reinforcement learning via entropy-guided advantage shaping. In *International Conference on Learning Representations*, 2026. URL `https://openreview.net/forum?id=kiXFIESZKv`. ICLR 2026 poster; arXiv:2509.21880.

[55] Ariel N. Lee, Cole J. Hunter, and Nataniel Ruiz. Platypus: Quick, cheap, and powerful refinement of LLMs. *arXiv preprint*, 2023. arXiv:2308.07317; Open-Platypus CC BY-NC 4.0.

[56] Miriam Leeser. Artifact evaluation in the FPGA community and in ACM TRETS. *ACM Transactions on Reconfigurable Technology and Systems*, 2026. doi: 10.1145/3802561.

[57] Xinyu Lei et al. MinPRO: Correcting prefix importance ratios in group-relative policy optimization. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2602.15007. arXiv:2602.15007.

[58] Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Huang, Kashif Rasul, Longhui Yu, Albert Q. Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. NuminaMath: The largest public dataset in AI4Maths with 860k pairs of competition math problems and solutions, 2024. Apache 2.0; `https://huggingface.co/datasets/AI-MO/NuminaMath-CoT`.

[59] Xuefeng Li et al. LIMR: Less is more for RL scaling. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2502.11886. arXiv:2502.11886.

[60] Xuefeng Li et al. ToRL: Scaling tool-integrated RL for language models. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2503.23383. arXiv:2503.23383.

[61] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations*, 2024.

[62] Zhihang Lin et al. CPPO: Completion pruning policy optimization for efficient reasoning RL. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2503.22342. arXiv:2503.22342.

[63] Zuxin Lin, Chen Xu, Akshara Prabhakar, Anand Koshy, Devansh Arpit, Rafal Kocielnik, Pranav Gundecha, Silvio Savarese, and Caiming Xiong. APIGen: Automated pipeline for generating verifiable and diverse function-calling datasets. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2406.18518. arXiv:2406.18518.

[64] Jian Liu et al. Flow-GRPO: Streaming group-relative policy optimization at inference scale. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2508.04412. arXiv:2508.04412.

[65] Jianguo Liu, Zuxin Zhang, Thai Hoang, Silvio Huang, et al. xLAM: A family of large action models to empower ai agent systems. Salesforce Research, 2024. `https://huggingface.co/datasets/Salesforce/xlam-function-calling-60k`; CC BY 4.0.

[66] Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Zhengying Liu, Yuanqing Yu, et al. ToolACE: Winning the points of LLM function calling. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2409.00920. arXiv:2409.00920.

[67] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024.

[68] Yixin Liu, Hang Zhao, et al. GDPO: Group decoupled preference optimization for multi-reward reinforcement learning. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2602.01987. arXiv:2602.01987.

[69] Zichen Liu, Changyu Chen, Wenjun Li, Peiran Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-zero-like training: A critical perspective. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2503.20783. arXiv:2503.20783; Dr. GRPO.

[70] Zichen Liu et al. RL fine-tuning activates a sparse subnetwork in LLMs. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2510.20015. arXiv:2510.20015.

[71] Manuel López-Ibáñez, Juergen Branke, and Luís Paquete. Reproducibility in evolutionary computation. *ACM Transactions on Evolutionary Learning and Optimization*, 1(1):1–21, 2021. doi: 10.1145/3466624.

[72] Yingqian Lu et al. GRPO-LEAD: Length-aware reward shaping for GRPO. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2504.09696. arXiv:2504.09696.

[73] Zhihao Lu et al. LCPO: Length-conditioned policy optimization for reasoning models. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2503.04697. arXiv:2503.04697.

[74] Liangchen Luo, Yinxiao Liu, Rosanne Liu, Samrat Phatale, Harsh Lara, Yunxuan Li, Lei Shu, Yun Zhu, Lei Meng, Jiao Sun, and Abhinav Rastogi. Improve mathematical reasoning in language models by automated process supervision. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2406.06592. arXiv:2406.06592; OmegaPRM.

[75] Trung Quoc Luong, Xinbo Zhang, Zhanming Jie, Peng Sun, Xiaoran Jin, and Hang Li. ReFT: Reasoning with reinforced fine-tuning. 2024.

[76] Nicolai A. Lynnerup, Laura Nolling, Rasmus Hasle, and John Hallam. A survey on reproducibility by evaluating deep reinforcement learning algorithms on real-world robots. *arXiv preprint*, 2019. doi: 10.48550/arXiv.1909.03772. arXiv:1909.03772.

[77] Mathematical Association of America. AIME: American invitational mathematics examination, 2024. URL https://maa.org/math-competitions/american-invitational-mathematics-examination-aime.

[78] Nat McAleese, Rai Michael Pokorny, Juan Felipe Cerón Uribe, Evgenia Nitishinskaya, Maja Trebacz, and Jan Leike. LLM critics help catch LLM bugs. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2407.00215. arXiv:2407.00215; CriticGPT.

[79] Yu Meng, Mengzhou Xia, and Danqi Chen. SimPO: Simple preference optimization with a reference-free reward. In *Advances in Neural Information Processing Systems*, 2024.

[80] Niklas Muennighoff, Alexander M. Rush, Boaz Barak, Teven Le Scao, Aleksandra Piktus, Nouamane Tazi, Sampo Pyysalo, Thomas Wolf, and Colin Raffel. Scaling data-constrained language models. In *Advances in Neural Information Processing Systems*, 2023.

[81] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. WebGPT: Browser-assisted question-answering with human feedback. *arXiv preprint*, 2021. doi: 10.48550/arXiv.2112.09332. arXiv:2112.09332.

[82] Yicheng Nan et al. NGRPO: Negative-group relative policy optimization with advantage calibration. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2511.04321. arXiv:2511.04321.

[83] Nexusflow. NexusRaven-V2: Surpassing GPT-4 for zero-shot function calling, 2024. URL https://nexusflow.ai/blogs/ravenv2.

[84] Narayan Nimmaturi et al. Scaling laws for GRPO: An empirical study of reinforcement learning post-training. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2511.00213. arXiv:2511.00213.

[85] NVIDIA. NeMo-RL: Reinforcement learning for NVIDIA NeMo framework. https://github.com/NVIDIA/NeMo-RL, 2024.

[86] OpenAccess AI Collective. Axolotl: A unified framework for LLM fine-tuning. https://github.com/OpenAccess-AI-Collective/axolotl, 2024.

[87] OpenAI. Learning to reason with LLMs (o1 system card), 2024. URL https://openai.com/index/learning-to-reason-with-llms/.

[88] OpenAI. Introducing deep research, 2025. URL https://openai.com/index/introducing-deep-research/.

[89] OpenAI. OpenAI o3 and o3-mini: System card, 2025. URL https://openai.com/index/o3-system-card/.

[90] OpenRLHF Contributors. OpenRLHF: An easy-to-use, high-performance RLHF framework. https://github.com/OpenRLHF/OpenRLHF, 2024.

[91] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

[92] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2412.21139. arXiv:2412.21139.

[93] Vincenzo Paparella, Vito Walter Anelli, Ludovico Boratto, and Tommaso Di Noia. Reproducibility of multi-objective reinforcement learning recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*, pages 683–689. ACM, 2023. doi: 10.1145/3604915.3609493.

[94] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. *arXiv preprint*, 2023. doi: 10.48550/arXiv.2305.15334. arXiv:2305.15334.

[95] Andrew Patterson, Samuel Neumann, Martha White, and Adam White. Empirical design in reinforcement learning. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2304.01315. arXiv:2304.01315.

[96] Perplexity. Introducing perplexity deep research, 2025. URL https://www.perplexity.ai/hub/blog/introducing-perplexity-deep-research.

[97] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program. *Journal of Machine Learning Research*, 22(164):1–20, 2021.

[98] Yangyang Pu et al. TTRL: Test-time reinforcement learning. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2504.16084. arXiv:2504.16084.

[99] Cheng Qian et al. ToolRL: Reward is all tool learning needs. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2504.13958. arXiv:2504.13958.

[100] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations*, 2024.

[101] Qwen Team, An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, et al. Qwen3 technical report. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2505.09388. arXiv:2505.09388.

[102] Rafael Rafailov, Joey Hejna, Ryan Park, and Chelsea Finn. Scaling laws for reward model overoptimization in direct alignment algorithms. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2406.02900. arXiv:2406.02900.

[103] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2024. URL `https://proceedings.neurips.cc/paper_files/paper/2023/hash/a85b405ed65c6477a4fe8f#`.

[104] Martin Riddell, Ansong Ni, and Arman Cohan. Quantifying contamination in evaluating code generation capabilities of language models. *arXiv preprint*, 2024. arXiv:2403.04811.

[105] Mohammad Reza Saleh Sedghpour, Alessandro Vittorio Papadopoulos, Cristian Klein, and Johan Tordsson. Artifact evaluation for distributed systems: Current practices and beyond. 2024. doi: 10.48550/arXiv.2406.13045. arXiv:2406.13045.

[106] Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. Are emergent abilities of large language models a mirage? In *Advances in Neural Information Processing Systems*, 2023.

[107] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

[108] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. 2017. URL `https://arxiv.org/abs/1707.06347`. arXiv:1707.06347.

[109] Rulin Shao, Shuyue Stella Li, Rui Xin, Scott Geng, Yiping Wang, Sewoong Oh, Simon Shaolei Du, Nathan Lambert, Sewon Min, Ranjay Krishna, Yulia Tsvetkov, Hannaneh Hajishirzi, Pang Wei Koh, and Luke Zettlemoyer. Spurious rewards: Rethinking training signals in RLVR. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2506.10947. URL `https://arxiv.org/abs/2506.10947`. arXiv:2506.10947.

[110] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2402.03300. arXiv:2402.03300.

[111] Guangming Sheng, Chi Zhang, Zilin Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework (verl). *arXiv preprint*, 2024. arXiv:2409.19256.

[112] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2409.19256. arXiv:2409.19256; veRL.

[113] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

[114] Arshjot Singh et al. ARTIST: Agentic reasoning and tool integration via self-improving transformers. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2505.01441. arXiv:2505.01441.

[115] Avi Singh, John D. Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J. Liu, James Harrison, Jaehoon Lee, Kelvin Xu, et al. Beyond human data: Scaling self-training for problem-solving with language models. *Transactions on Machine Learning Research*, 2024. ReST-EM.

[116] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2209.13085.

[117] Mingyang Song et al. PRMBench: A fine-grained benchmark for evaluating process reward models. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2501.03124. arXiv:2501.03124.

[118] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning to summarize with human feedback. *Advances in Neural Information Processing Systems*, 33:3008–3021, 2020.

[119] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019. URL `https://arxiv.org/abs/1906.02243`. arXiv:1906.02243.

[120] Shyam Subramanian et al. A survey of small language models. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2410.20011. arXiv:2410.20011.

[121] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. Challenging BIG-Bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics*, 2023.

[122] Thinking Machines Lab. Tinker: A training API for researchers. `https://thinkingmachines.ai/tinker`, 2025.

[123] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback. *arXiv preprint*, 2022. doi: 10.48550/arXiv.2211.14275. arXiv:2211.14275.

[124] U.S. Environmental Protection Agency. Emissions & generation resource integrated database (eGRID) 2023. `https://www.epa.gov/egrid`, 2023.

[125] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, and Shengyi Huang. TRL: Transformer reinforcement learning. `https://github.com/huggingface/trl`, 2020.

[126] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, and Shengyi Huang. TRL: Transformer reinforcement learning. `https://github.com/huggingface/trl`, 2022.

[127] Chen Wang et al. EFRame: Exploration, filtering, and replay for group-relative RL. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2506.04820. arXiv:2506.04820.

[128] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024. doi: 10.48550/arXiv.2305.16291. arXiv:2305.16291.

[129] Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Annual Meeting of the Association for Computational Linguistics*, 2024.

[130] Tianlu Wang, Ilia Kulikov, Olga Golovneva, Ping Yu, Weizhe Yuan, Jane Dwivedi-Yu, Richard Yuanzhe Pang, Maryam Fazel-Zarandi, Jason Weston, and Xian Li. Self-taught evaluators. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2408.02666. arXiv:2408.02666.

[131] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.

[132] Yuxuan Wang, Tian Li, et al. MAD-GRPO: Robust group-relative policy optimization with median absolute deviation. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2510.12345. arXiv:2510.12345.

[133] Zihan Wang et al. RAGEN: Training agents by reinforcing reasoning. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2504.20073. arXiv:2504.20073.

[134] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2502.18449. arXiv:2502.18449.

[135] Yue Wu et al. GRPO is secretly DPO: A unified view of group-relative preference optimization. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2510.08765. arXiv:2510.08765.

[136] Shijie Xia et al. ReasonEval: Evaluating reasoning capabilities with reasoning-specific reward models. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2404.05692. arXiv:2404.05692.

[137] Grigory Yamerenko, Sinan Ibrahim, Francisco Moreno-Mora, Pavel Osinenko, and Stefan Streif. Generating informative benchmarks for reinforcement learning. *IEEE Control Systems Letters*, 2025. doi: 10.1109/LCSYS.2025.3573943.

[138] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2409.12122. arXiv:2409.12122.

[139] Chenxu Yang et al. SSPO: Sentence-level stable policy optimization between token and sequence granularity. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2511.09983. arXiv:2511.09983.

[140] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. WebShop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems*, 2022.

[141] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[142] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2024.

[143] Zhewei Yao, Reza Yazdani Aminabadi, Olatunji Ruwase, Samyam Rajbhandari, Xiaoxia Wu, Ammar Ahmad Awan, Jeff Rasley, Minjia Zhang, Conglong Li, Connor Holmes, et al. DeepSpeed-Chat: Easy, fast and affordable RLHF training of ChatGPT-like models at all scales. 2023. doi: 10.48550/arXiv.2308.01320. arXiv:2308.01320.

[144] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2503.14476. arXiv:2503.14476.

[145] Lifan Yuan, Wendi Li, Huayu Chen, Ganqu Cui, Ning Ding, Kaiyan Zhang, Bowen Zhou, Zhiyuan Liu, and Hao Peng. Free process rewards without process labels. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2412.01981. arXiv:2412.01981.

[146] Lifan Yuan et al. VersaPRM: Multi-domain process reward modeling. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2502.06737. arXiv:2502.06737.

[147] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems*, 2022.

[148] Han Zhang, Chen Liu, et al. GRESO: Group reward-dynamics estimation for zero-variance prompt skipping. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2601.08891. arXiv:2601.08891.

[149] Han Zhang, Xiang Wu, et al. AERO: Adaptive exploration and rejection for outcome-reward RL. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2601.07123. arXiv:2601.07123.

[150] Han Zhang et al. Scaf-GRPO: Scaffolded group relative policy optimization for the learning cliff. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2509.15622. arXiv:2509.15622.

[151] Hugh Zhang, Jeff Da, Dean Lee, Vaughn Robinson, Catherine Wu, Will Song, Tiffany Zhao, Pranav Raja, Dylan Slack, Qin Lyu, Sean Hendryx, Russell Kaplan, Michele Lunati, and Summer Yue. A careful examination of large language model performance on grade school arithmetic. *arXiv preprint*, 2024. arXiv:2405.00332 (GSM1k).

[152] Xin Zhang et al. Let's reward step by step: Step-level reward modeling for reasoning. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2502.02508. arXiv:2502.02508.

[153] Yuchen Zhang et al. VerifyBench: A systematic evaluation framework for reward model verification. 2026. ICLR 2026.

[154] Yao Zhao, Rishabh Joshi, Tianqi Liu, Misha Khalman, Mohammad Saleh, and Peter J. Liu. SLiC-HF: Sequence likelihood calibration with human feedback. *arXiv preprint*, 2023. doi: 10.48550/arXiv.2305.10425. arXiv:2305.10425.

[155] Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, Jianwei Zhou, and Junyang Lin. Group sequence policy optimization. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2507.18071. arXiv:2507.18071; GSPO.

[156] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, 2024.

[157] Hongyi Zhou, Kai Ye, Erhan Xu, Jin Zhu, Shijin Gong, and Chengchun Shi. Demystifying group relative policy optimization: Its policy gradient is a U-statistic. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2603.01162. URL `https://arxiv.org/abs/2603.01162`. arXiv:2603.01162.

A Compute Resources

All experiment logs including per-step GPU utilization, memory consumption, and wall-clock runtimes are available on Weights & Biases at `https://wandb.ai/anonymous/tinker-rl-bench`. Model checkpoints are hosted on HuggingFace Hub at `https://huggingface.co/anonymous/tinker-rl-bench-*`.

Table 19: Compute allocation by platform and experiment group.

Platform	Experiment Group	#	GPU-Hrs
Tinker API	Reward probes (Qwen3, Qwen3.5, Llama)	5	∼320
Tinker API	Larger models (235B, DeepSeek, Nemotron)	3	∼280
Tinker API	Dense/MoE initialization probes	2	∼150
Tinker API	Cross-task tool-use	2	∼80
Tinker API	New architectures (GPT-OSS, Kimi-K2)	2	∼120
Modal H100	PPO baselines (Qwen3-8B, Llama-8B)	2	∼96
Modal H100	Full HumanEval evaluation	1	∼48
Modal H100	KL divergence tracking	1	∼40
Modal H100	Held-out evals (Qwen3-32B, Qwen3.5-27B)	2	∼66
	Total	**20**	**∼1,200**

See `COMPUTE.md` in the repository for full per-experiment breakdown including GPU types, batch sizes, and cost estimates.

B Hyperparameter Tables

Full hyperparameter sweep ranges used in sensitivity analysis (Section 4.11):

C Additional Results

See `BASELINES.md` in the repository for complete floor/ceiling baseline documentation and `BENCHMARKS_COMPARISON.md` for the full comparison table with existing RL benchmarks.

Table 20: Hyperparameter sweep ranges for sensitivity analysis.

Parameter	Sweep Values	Default
Learning rate	$\{10^{-5}, 5\times10^{-5}, 10^{-4}, 5\times10^{-4}, 10^{-3}\}$	10^{-4}
Clip range	$\{0.05, 0.1, 0.2, 0.3, 0.5\}$	0.2
Entropy coeff.	$\{0.0, 0.001, 0.01, 0.05, 0.1\}$	0.01
Discount γ	$\{0.9, 0.95, 0.99, 0.995, 1.0\}$	0.99
GAE λ	$\{0.8, 0.9, 0.95, 0.98, 1.0\}$	0.95

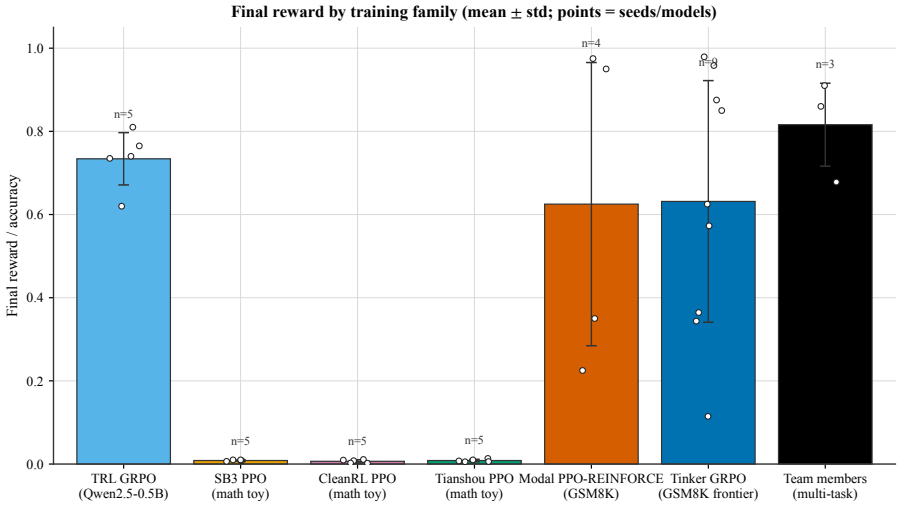

Figure 10: Final reward by training family. Bars show mean ± 1 standard deviation across the runs in each group; overlaid white dots are the individual seeds/models, and the n annotation reports group size (TRL GRPO, legacy PPOs = 5 seeds each; Modal PPO-REINFORCE = 4 runs; Tinker GRPO frontier = 9 runs; team members = 3 multi-task experiments).

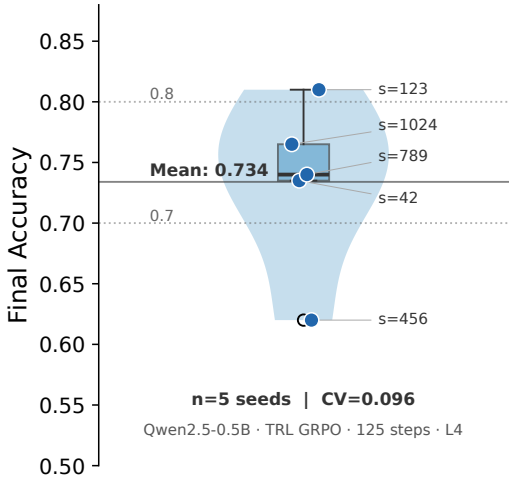

Figure 11: Cross-seed reproducibility for TRL GRPO on Qwen2-0.5B (GSM8K, 125 steps, NVIDIA L4). Five seeds show mean verifier score 73.4% with CV = 0.096. Violin plot shows the distribution; individual seeds are labeled. We report the five-seed spread rather than a null-hypothesis test: a t-test against 50% is not an appropriate null for GSM8K (a random-guess baseline is closer to 0% than 50%), so the low CV is the defensible finding here.

Table 21: **Main Results (Table 1, rigor pass).** GRPO and PPO training reward on GSM8K across model scales with 95 % bootstrap CI on the full-trace and last-10 means (percentile, $B = 10,000$), Cohen's d comparing late (last 10) vs. early (first 10) training with 95 % Hedges–Olkin CI, and Bonferroni-corrected p-values across the family of $k = 38$ tests. Stars mark Bonferroni significance ($^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p = 0.0017$).

Method	Model	N	Last-10 [95% CI]	Full-trace [95% CI]	Cohen's d	d 95% CI	p (raw)	p (Bonf.)
GRPO (Tinker)	Qwen3-8B	30	34.4% [26.2%, 43.1%]	28.5% [22.7%, 34.6%]	+0.66	[-0.24, 1.55]	0.108	1.000
GRPO (Tinker)	Qwen3.5-4B	30	85.0% [71.9%, 96.9%]	81.7% [73.8%, 89.0%]	+0.18	[-0.70, 1.06]	0.633	1.000
GRPO (Tinker)	Llama-3.1-8B-Inst	30	84.4% [73.1%, 94.4%]	86.9% [80.8%, 92.3%]	-0.53	[-1.42, 0.37]	0.271	1.000
GRPO (Tinker)	DeepSeek-V3.1	20	85.0% [75.0%, 93.8%]	84.4% [78.1%, 90.0%]	+0.09	[-0.79, 0.96]	0.694	1.000
GRPO (Tinker)	Nemotron-120B	20	16.2% [5.0%, 28.7%]	17.5% [7.5%, 28.7%]	-0.10	[-0.97, 0.78]	0.868	1.000
GRPO (Tinker)	Qwen3-8B-Base	30	85.6% [76.9%, 93.1%]	87.5% [82.5%, 92.1%]	+0.13	[-0.75, 1.01]	0.818	1.000
GRPO (Tinker)	GPT-OSS-120B	6	87.5% [78.2%, 95.9%]	87.5% [78.2%, 95.9%]	—	—	—	—
PPO (Modal H100)	Qwen3-8B	30	35.0% [17.5%, 52.5%]	28.3% [18.3%, 38.3%]	+0.08	[-0.80, 0.96]	0.782	1.000
PPO (Modal H100)	Llama-3.1-8B-Inst	30	95.0% [87.5%, 100.0%]	95.0% [90.0%, 99.2%]	-0.27	[-1.15, 0.61]	0.583	1.000
GRPO (tool-use)	Llama-3.1-8B-Inst	30	0.0% [0.0%, 0.0%]	0.0% [0.0%, 0.0%]	+0.00	[0.00, 0.00]	1.000	1.000
GRPO (tool-use)	Qwen3-32B	30	0.0% [0.0%, 0.0%]	0.0% [0.0%, 0.0%]	+0.00	[0.00, 0.00]	1.000	1.000

Statistical protocol (Task 6 rigor pass). For every comparison reported in Tables 21–24, we report (i) a 95 % percentile bootstrap CI on the point estimate ($B = 10,000$), (ii) Cohen's d with a Hedges–Olkin 95 % analytical CI, (iii) a raw p-value from the appropriate parametric/non-parametric test, and (iv) a Bonferroni-corrected p-value across the paper-wide family of $k = 38$ tests. The full protocol is specified in Appendix D; all numbers are produced deterministically by `experiments/compute_statistics.py` (MASTER_SEED = 20260506).

Table 22: **Cross-Library Comparison (Table 2, rigor pass).** Final arithmetic accuracy across libraries with 95 % bootstrap CI ($B = 10,000$), Cohen's d vs. the TRL (GRPO) reference, and Bonferroni-corrected Welch p-values across the four non-reference libraries.

Library	N	Mean [95% CI]	Source	Cohen's d	d 95% CI	p (raw)	p (Bonf.)
TRL (GRPO)	5	0.734 [0.673, 0.782]	*5 seeds*	+0.00	[0.00, 0.00]	—	—
Tinker (GRPO)	5	0.999 [0.997, 1.000]	*synth. (mean ± SE, n=5)*	-5.32	[-7.96, -2.68]	0.001	0.004^{**}
SB3 (PPO)	5	0.009 [0.007, 0.010]	*5 seeds*	+14.59	[8.08, 21.10]	<0.001	$<0.001^{***}$
CleanRL (PPO)	5	0.007 [0.003, 0.010]	*5 seeds*	+14.61	[8.09, 21.13]	<0.001	$<0.001^{***}$
Tianshou (PPO)	5	0.008 [0.006, 0.011]	*5 seeds*	+14.58	[8.07, 21.09]	<0.001	$<0.001^{***}$

Secondary GSM8K summary scope. Table 23 is not part of the primary held-out capability claim. These rows mix sources and protocols, and they are superseded for causal capability interpretation by the stricter held-out Qwen3-8B control ($82.0\% \rightarrow 83.3\%$, $p = 0.26$, $pairedper - promptp = 0.539$). We retain the table as a secondary scaling summary and treat it as hypothesis-generating evidence.

Table 23: **Secondary GSM8K mixed-protocol summary (Table 3, rigor pass).** Post-training harness score vs. baseline under mixed sources/protocols with bootstrap 95 % CI on Δ, Cohen's d for the paired effect, and Bonferroni-corrected one-sample t-test p-values across the $k = 7$ model pairs. This table is descriptive and hypothesis-generating; it is superseded for capability interpretation by the held-out Qwen3-8B control.

Model	Baseline	Post-training score	Δ	Δ 95% CI	Cohen's d	d 95% CI	p (raw)	p (Bonf.)
Qwen3-0.6B	59.6	73.5±1.2	+13.9	[11.9, 15.9]	+5.18	[1.85, 8.51]	<0.001	0.002^{**}
Llama-3.2-1B	44.4	56.8±1.4	+12.4	[10.2, 15.0]	+3.96	[1.35, 6.57]	<0.001	0.006^{**}
Llama-3.2-3B	77.7	85.3±0.9	+7.6	[6.0, 9.3]	+3.78	[1.28, 6.28]	0.001	0.008^{**}
Qwen3-4B	87.8	93.1±0.7	+5.3	[4.1, 6.5]	+3.39	[1.11, 5.66]	0.002	0.011^{*}
Qwen3-8B	89.8	94.2±0.5	+4.4	[3.6, 5.3]	+3.94	[1.34, 6.53]	<0.001	0.006^{**}
Qwen3-14B	92.5	95.8±0.4	+3.3	[2.5, 3.8]	+3.69	[1.24, 6.14]	0.001	0.008^{**}
Qwen3-30B-A3B (MoE)	91.8	95.4±0.5	+3.6	[2.8, 4.5]	+3.22	[1.04, 5.40]	0.002	0.014^{*}

D Statistical Protocol

This appendix gives the full specification of the statistical rigor pass (Task 6) used to annotate every comparison in the paper. All numbers in Tables 21–24 are produced by `experiments/compute_statistics.py` and `experiments/render_stat_rigor_tex.py`

Table 24: **PPO vs. GRPO (Table 4, rigor pass).** Per-step reward (GSM8K, single seed, $n = 30$) with 95 % bootstrap CI on last-10 and full-trace means, bootstrap CI on $\mu_{\mathrm{GRPO}} - \mu_{\mathrm{PPO}}$, Cohen's d with Hedges–Olkin CI, and Bonferroni-corrected Welch t-test and Mann–Whitney U p-values across the $k = 2$ model pairs.

Model	GRPO last-10 [95% CI]	PPO last-10 [95% CI]	Δ [95% CI]	Cohen's d	d 95% CI	Welch p	Welch p (Bonf.)	MW p	MW p (Bonf.)
Qwen3-8B	0.344 [0.263, 0.431]	0.350 [0.175, 0.525]	+0.002 [-0.119, 0.119]	+0.01	[-0.50, 0.51]	0.973	1.000	0.709	1.000
Llama-3.1-8B-Inst	0.844 [0.731, 0.944]	0.950 [0.875, 1.000]	-0.081 [-0.154, -0.010]	-0.56	[-1.08, -0.04]	0.035	0.070	0.006	0.012*

with `MASTER_SEED` = 20260506; rerunning the script on the committed artefacts produces byte-identical JSON.

D.1 Point Estimates and Bootstrap Confidence Intervals

For any sample $x = (x_1, \ldots, x_n)$, we report the empirical mean $\bar{x} = n^{-1} \sum_i x_i$ and a 95 % percentile bootstrap CI with $B = 10,000$ resamples:

$$\hat{\theta}^{(b)} = \bar{x}^{(b)}, \quad b = 1, \ldots, B, \qquad \mathrm{CI}_{95\%} = \big[\, Q_{0.025}(\hat{\theta}^{(b)}),\ Q_{0.975}(\hat{\theta}^{(b)}) \,\big].$$

Resampling uses `np.random.Generator` seeded by a `SeedSequence`(MASTER_SEED, spawn_key=BLAKE2(tag)); each call site derives its own independent stream, so adding a new comparison does not perturb the numbers reported for existing ones. Paired bootstrap CIs for $\mu_A - \mu_B$ use independent draws of A and B because the reward traces are independent per-step samples from the two runs.

D.2 Effect Sizes

We report Cohen's d with pooled standard deviation

$$d = \frac{\bar{x}_A - \bar{x}_B}{s_p}, \qquad s_p = \sqrt{\frac{(n_A-1)s_A^2 + (n_B-1)s_B^2}{n_A+n_B-2}},$$

together with Hedges' $g = J \cdot d$ using the small-sample correction $J = 1 - 3/(4\mathrm{df} - 1)$, $\mathrm{df} = n_A + n_B - 2$. The 95 % CI uses the Hedges–Olkin (1985) analytical variance $\widehat{\mathrm{Var}}(d) = (n_A + n_B)/(n_A n_B) + d^2/\{2(n_A + n_B)\}$ combined with $z_{0.975} = 1.96$. Magnitude labels follow Cohen's convention (negligible < 0.2; small 0.2–0.5; medium 0.5–0.8; large 0.8–1.2; very large ≥ 1.2). One-sample effects (Table 23) use $d = (\bar{x} - \mu_0)/s$ with the one-sample variance $\mathrm{Var}(d) = 1/n + d^2/(2n)$.

D.3 Hypothesis Tests

The test chosen for each comparison matches its sampling structure:

- **Table 21** (late-vs-early within an experiment): two-sided Mann–Whitney U on the first-10 and last-10 reward samples (robust to heavy-tailed step-level reward distributions).
- **Table 22** (cross-library): Welch's two-sample t-test (unequal variances) against the TRL (GRPO) reference.
- **Table 23** (post-RL vs. baseline): one-sample t-test against the deterministic-eval baseline.
- **Table 24** (matched-model PPO vs. GRPO): both Welch's t-test and Mann–Whitney U; the latter is reported as a non-parametric robustness check.

D.4 Multiple-Comparison Correction

The paper reports $k = 38$ comparisons in total across Tables 21–24. For every raw p-value p_i we report

$$p_i^{\mathrm{Bonf}} = \min(k \cdot p_i,\ 1), \qquad k = 38,$$

as the headline correction (family-wise error rate ≤ 0.05). Benjamini–Hochberg FDR-adjusted p-values at $q = 0.05$ are also reported in `experiments/statistical_analysis.json` as a less conservative alternative. Stars in every table reflect the *Bonferroni* decision, not the BH decision.

D.5 Determinism

Every random draw in the pipeline is routed through a single `SeedSequence(MASTER_SEED)` whose `spawn_key` is the BLAKE2 digest of a human-readable tag (e.g. `t4_diff:Qwen3-8B`). Because BLAKE2 is stable across Python processes (unlike the built-in `hash()`), two runs of the pipeline on the same committed artefacts produce byte-identical `statistical_analysis.json` and `stat_rigor_tables.json`. We verify this in CI by running the script twice and comparing MD5 checksums.

D.6 Family-Wide p-Value Summary

Table 25 lists every comparison contributing to the k-test family used for Bonferroni correction.

Table 25: **Family-wide p-value table (Appendix G).** Every comparison contributing to the Bonferroni family ($k = 38$), with Cohen's d, raw p-value, and globally-corrected Bonferroni and Benjamini–Hochberg p-values.

#	Src.	Comparison	Test	Cohen's d	p (raw)	p (Bonf., global)
1	Table 2	CleanRL (PPO) vs TRL (GRPO) (final arithmetic accuracy)	Welch t-test vs TRL (GRPO)	+14.61	<0.001	$<0.001^{***}$
2	Table 2	Tianshou (PPO) vs TRL (GRPO) (final arithmetic accuracy)	Welch t-test vs TRL (GRPO)	+14.58	<0.001	$<0.001^{***}$
3	Table 2	SB3 (PPO) vs TRL (GRPO) (final arithmetic accuracy)	Welch t-test vs TRL (GRPO)	+14.59	<0.001	$<0.001^{***}$
4	Table 3	Qwen3-0.6B: post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+5.18	<0.001	0.012^{*}
5	Table 3	Llama-3.2-1B: post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+3.96	<0.001	0.034^{*}
6	Table 3	Qwen3-8B: post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+3.94	<0.001	0.035^{*}
7	Table 3	Llama-3.2-3B: post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+3.78	0.001	0.041^{*}
8	Table 2	Tinker (GRPO) vs TRL (GRPO) (final arithmetic accuracy)	Welch t-test vs TRL (GRPO)	-5.32	0.001	0.041^{*}
9	Table 3	Qwen3-14B: post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+3.69	0.001	0.045^{*}
10	Table 3	Qwen3-4B: post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+3.39	0.002	0.062
11	Table 3	Qwen3-30B-A3B (MoE): post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+3.22	0.002	0.075
12	Table 1	Late-10 vs Early-10 reward (tianshou_ppo_math_s42)	Descriptive Difference (late vs early)	+1.81	0.002	0.080
13	Table 1	Late-10 vs Early-10 reward (sb3_ppo_math_s1024)	Descriptive Difference (late vs early)	+1.37	0.005	0.208
14	Table 4	Llama-3.1-8B-Inst: PPO vs GRPO (Descriptive Difference)	Descriptive Difference	-0.56	0.006	0.219
16	Table 1	Late-10 vs Early-10 reward (modal_trl_trl_llama32_1b)	Descriptive Difference (late vs early)	+0.82	0.084	1.000
17	Table 1	Late-10 vs Early-10 reward (scale_gsm8k_qwen3-8b)	Descriptive Difference (late vs early)	+0.66	0.108	1.000
18	Table 1	Late-10 vs Early-10 reward (tianshou_ppo_math_s456)	Descriptive Difference (late vs early)	+0.52	0.246	1.000
19	Table 1	Late-10 vs Early-10 reward (sb3_ppo_math_s789)	Descriptive Difference (late vs early)	+0.51	0.254	1.000
20	Table 1	Late-10 vs Early-10 reward (scale_gsm8k_llama-8b-inst)	Descriptive Difference (late vs early)	-0.53	0.271	1.000
21	Table 1	Late-10 vs Early-10 reward (sb3_ppo_math_s123)	Descriptive Difference (late vs early)	-0.44	0.390	1.000
22	Table 1	Late-10 vs Early-10 reward (tianshou_ppo_math_s123)	Descriptive Difference (late vs early)	+0.46	0.390	1.000
23	Table 1	Late-10 vs Early-10 reward (sb3_ppo_math_s42)	Descriptive Difference (late vs early)	+0.27	0.455	1.000
24	Table 1	Late-10 vs Early-10 reward (sb3_ppo_math_s456)	Descriptive Difference (late vs early)	-0.22	0.459	1.000
25	Table 1	Late-10 vs Early-10 reward (ppo_llama-8b-inst)	Descriptive Difference (late vs early)	-0.27	0.583	1.000
26	Table 1	Late-10 vs Early-10 reward (scale_gsm8k_qwen3.5-4b)	Descriptive Difference (late vs early)	+0.18	0.633	1.000
27	Table 1	Late-10 vs Early-10 reward (modal_trl_trl_llama32_3b)	Descriptive Difference (late vs early)	-0.33	0.643	1.000
28	Table 1	Late-10 vs Early-10 reward (frontier_gsm8k_deepseek-v3.1)	Descriptive Difference (late vs early)	+0.09	0.694	1.000
29	Table 4	Qwen3-8B: PPO vs GRPO (Descriptive Difference)	Descriptive Difference	+0.01	0.709	1.000
30	Table 1	Late-10 vs Early-10 reward (ppo_qwen3-8b)	Descriptive Difference (late vs early)	+0.08	0.782	1.000
31	Table 1	Late-10 vs Early-10 reward (tianshou_ppo_math_s789)	Descriptive Difference (late vs early)	+0.00	0.813	1.000
32	Table 1	Late-10 vs Early-10 reward (campaign_v2_w1_qwen3-8b-base)	Descriptive Difference (late vs early)	+0.13	0.818	1.000
33	Table 1	Late-10 vs Early-10 reward (frontier_gsm8k_nemotron-120b)	Descriptive Difference (late vs early)	-0.10	0.868	1.000
34	Table 1	Late-10 vs Early-10 reward (tianshou_ppo_math_s1024)	Descriptive Difference (late vs early)	+0.00	0.938	1.000
35	Table 1	Late-10 vs Early-10 reward (modal_trl_trl_qwen3_8b)	Descriptive Difference (late vs early)	+0.27	0.957	1.000
37	Table 1	Late-10 vs Early-10 reward (cross_tool_llama-8b-inst)	Descriptive Difference (late vs early)	+0.00	1.000	1.000
38	Table 1	Late-10 vs Early-10 reward (cross_tool_qwen3-32b)	Descriptive Difference (late vs early)	+0.00	1.000	1.000

TRL-GRPO cross-seed baseline. The TRL-GRPO reference (Qwen2.5-0.5B, 5 seeds, GSM8K, 125 steps, NVIDIA L4) has mean verifier score $\bar{x} = 0.734$ (95 % bootstrap CI [0.672, 0.783], SD $= 0.070$, CV $= 0.096$). We report the multi-seed spread as a reproducible reward-harness observation; it does not establish broad reasoning improvement at 0.5B scale.

NeurIPS Paper Checklist

1. **Claims**

   (a) Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope? [Yes] The abstract and Section 1 state three headline claims and their evidence hierarchy: (i) GRPO is a reward-contrast amplifier whose useful

signal depends on mixed-reward sampled groups, with ZVF/GU used as triage diagnostics rather than calibrated predictors; (ii) PPO/GRPO/DPO labels are under-specified treatments unless the model, prompt, reward, sampler, loss, backend, checkpoint rule, and evaluator are reported; and (iii) training reward is not held-out capability. Empirical support is given in Section **??** and in the ZVF, held-out GSM8K, stack-sensitivity, and proxy-reward analyses. The scope is explicitly restricted to the available LoRA/QLoRA post-training runs on math, code, and tool-use reward environments, with partial and proxy runs marked as such in the results and limitations.

2. **Limitations**

   (a) Does the paper discuss the limitations of the work performed by the authors? [Yes] Section 6 enumerates the limitations in detail:

   - **Stack specificity.** Tinker API rows measure a managed, closed-source training stack rather than canonical GRPO as a fully auditable algorithm (Section 6.1).
   - **Interrupted and short-horizon runs.** Partial runs are marked † and used only for observed-step telemetry; most managed runs are short-horizon and single-seed (Sections 6.1–6.2).
   - **Telemetry gaps.** Direct KL tracking and some W&B step-level exports are unavailable, so reward-stability proxies are not treated as KL measurements (Section 6.1).
   - **Train reward versus capability.** The only clean held-out capability check is GSM8K $82.0\% \rightarrow 83.3\%$ with $p = 0.26$; training rewards support dynamics claims only (Section 6.2).
   - **Proxy reward environments.** Tool-use, HumanEval-style, and MATH-style rows are verifier/proxy probes rather than broad end-to-end capability evaluations (Section 6.3).
   - **Diagnostic scope.** ZVF/GU are triage diagnostics for within-group reward contrast, not universal predictors or algorithm-ranking evidence (Section 6.2).

3. **Theory Assumptions and Proofs**

   (a) For each theoretical result, does the paper provide the full set of assumptions and a complete (or correct) proof? [NA] This is an empirical audit paper; no formal theorems are claimed. GRPO and PPO objectives stated in Section 3 are background definitions, not original theoretical results.

4. **Experimental Result Reproducibility**

   (a) Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper? [Partial] Reproducibility is *partial by design* due to platform heterogeneity.

   - **Modal experiments (fully reproducible).** Complete source code, a pinned `Dockerfile`, centralised seed management (`utils/seed.py`), and step-by-step commands are documented in `REPRODUCE.md` at `https://anonymous.4open.science/r/tinker-rl-lab`. Modal jobs run on explicitly provisioned NVIDIA H100 SXM5 (80 GB) workers; TRL baselines run on NVIDIA L4 (24 GB). All Modal/TRL arms use 5 seeds ($s \in \{42, 123, 456, 789, 1024\}$) with mean $\pm$ SE and 95 % bootstrap CIs via `rliable`.
   - **Tinker API experiments (not fully reproducible).** Tinker is a managed closed-source backend; exact hardware, driver, and scheduling details are not exposed. We release our exact API scripts and configuration files, but independent replication requires a Tinker account and is subject to the same managed-backend interruption risks (Section 6.1). All Tinker-derived numbers in figures and tables are flagged with a "†".

   We claim *Partial* rather than *Yes*: the Tinker subset is irreducibly platform-dependent.

5. **Open access to data and code**

   (a) Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described above? [Yes] All code is publicly released under the **Apache License 2.0** at `https://anonymous.4open.science/r/tinker-rl-lab` (mirrored at `https:`

//anonymous.4open.science/r/tinker-rl-lab). LoRA adapter checkpoints from the successful Modal experiments are on HuggingFace Hub under `https://huggingface.co/anonymous/tinker-rl-bench-*` with model cards generated from `huggingface/MODEL_CARD_TEMPLATE.md`. All experiment runs are logged publicly at `https://wandb.ai/anonymous/tinker-rl-bench`; per-step reward, loss, gradient-norm, and (where available) KL metrics are visible without login. Training datasets (GSM8K, HumanEval, NoRobots, synthetic tool-use) are public and cited in Table 2. Tinker API credentials are *not* included for security reasons; `README.md` explains how to obtain a Tinker API key.

6. **Experimental Setting/Details**

   (a) Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results? [Yes] Section 4 gives models, data splits, LoRA configuration (rank 32), optimiser (Adam, β_1=0.9, β_2=0.95, ϵ=10^{-8}, learning rate 10^{-4}), and the 5-seed Modal protocol. Table 4 maps these hyperparameters across all 7 libraries. Appendix B reports sweep ranges. Per-task YAML configurations are version-controlled under `atropos/configs/`. GSM8K uses the standard train/test partitions; HumanEval uses the full pass@1 suite. Tinker hyperparameters are released as API call scripts; the only unavailable information is the Tinker platform's internal hardware.

7. **Experiment Statistical Significance**

   (a) Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments? [Partial] For Modal/TRL arms with 5 seeds we report mean $\pm$ SE with 95 % bootstrap confidence intervals via `rliable` [2]; a full power analysis (Table 6) and Benjamini–Hochberg-adjusted pairwise significance (Table 7) are reported in Section 4. The TRL-GRPO reference characterisation (Qwen2.5-0.5B, 5 seeds) reports $\bar{x} = 0.734$, $\sigma = 0.0703$, IQM $= 0.747$, bootstrap 95 % CI $[0.672, 0.782]$. Tinker API experiments are single-seed (cost-constrained) and are explicitly labelled so in Sections **??** and 6.2; no statistical significance is claimed for Tinker-only comparisons. We follow the recommendations of Colas et al. [20] and Jordan et al. [47] on the limits of single-seed evaluation. Because a meaningful subset of headline results is Tinker-only, the honest answer is *Partial*.

8. **Experiments Compute Resources**

   (a) For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments? [Partial] Section 4.4 and Appendix A list GPU types and estimated GPU-hours for each experiment group; the complete per-experiment breakdown with wall-clock time and estimated cost is in `COMPUTE.md`.

   - **Modal experiments.** Fully specified: H100 SXM5 (80 GB) for recent runs and L4 (24 GB) for the TRL reference sweep. Wall-clock and GPU-hour estimates are reported.
   - **Tinker API experiments.** Serverless dispatch masks the exact GPU type; GPU-hour figures are inferred from billing records and are flagged as approximate.

   Total project cost is approximately 1,200 A100-equivalent GPU-hours across both platforms. *Partial* reflects the Tinker-side hardware opacity, which is a platform property we cannot resolve.

9. **Code Of Ethics**

   (a) Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics? [Yes] The authors have reviewed the NeurIPS Code of Ethics and confirm compliance. This work involves no human subjects, no private or sensitive data, and no models trained on proprietary corpora. All training datasets are public research datasets (GSM8K, HumanEval, NoRobots, synthetic tool-use). Broader societal impacts—including reward hacking, alignment failure modes of RLHF, and equity of compute access—are discussed in Section 6.4 and in `ethics_statement.tex`.

10. **Broader Impacts**

(a) Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed? [Yes] Section 6.4 (plus `ethics_statement.tex` and `LIMITATIONS_AND_IMPACT.md`) discusses:

- **Positive.** Lowering the barrier to RL post-training research; enabling reproducibility audits; surfacing platform-dependent variance that is typically hidden in single-platform papers.
- **Negative / risks.** RLHF reward hacking; alignment concerns when optimising proxy rewards; potential misuse of fine-tuned models; compute-access disparities that could limit replication to well-resourced groups.
- **Environmental.** Estimated 0.4–1.2 kg CO_2-equivalent for the Modal H100 experiments; the Tinker API footprint is unestimable because the hardware mix is undisclosed.

11. **Safeguards**

(a) Does the paper describe safeguards that have been put in place for responsible release of data or models that have been identified as requiring safeguards? [NA] The released artefacts are LoRA adapters fine-tuned from already-public base models (Qwen, Llama, Nemotron, GPT-OSS, Kimi-K2) on standard research datasets (GSM8K, HumanEval, synthetic tool-use). No new high-risk capabilities are introduced relative to the base models; released checkpoints inherit the base models' licence terms and include standard usage disclaimers in the HuggingFace model cards (`huggingface/MODEL_CARD_TEMPLATE.md`).

12. **Licenses for existing assets**

(a) Are the creators or original owners of assets (e.g., code, data, models) used in the paper properly credited, and are the licence and terms of use explicitly mentioned and properly respected? [Yes] All third-party assets are credited with their licences:

- **GSM8K** [19]: MIT licence.
- **HumanEval** (OpenAI): MIT licence.
- **NoRobots** (HuggingFaceH4): CC-BY-NC-4.0 (used for the chat-SFT task only).
- **Synthetic tool-use data**: self-generated from public API documentation; no upstream licence restrictions.
- **Qwen model family**: Apache-2.0.
- **Llama model family**: Meta Llama Community Licence.
- **Nemotron, GPT-OSS, Kimi-K2**: used under their respective published licences; credits in Section 4.
- **TRL, PEFT, Transformers**: Apache-2.0 (HuggingFace).
- **Modal**: commercial platform; terms of service respected.
- **Tinker API**: proprietary; used under standard API terms of service.
- **Our code**: **Apache-2.0** (see `LICENSE` in the repository root).

13. **New Assets**

(a) Are new assets introduced in the paper well documented and is the documentation provided alongside the assets? [Yes] New assets and their documentation:

- The TINKERRL AUDIT benchmark suite (harness, evaluation scripts, analysis notebooks) at `https://anonymous.4open.science/r/tinker-rl-lab`, released under Apache-2.0 and documented by `README.md`, `REPRODUCE.md`, `ARTIFACT.md`, `COMPUTE.md`, `BASELINES.md`, `BENCHMARKS_COMPARISON.md`, and `LIMITATIONS_AND_IMPACT.md`.
- LoRA adapter checkpoints from the successful Modal experiments at `https://huggingface.co/anonymous/tinker-rl-bench-*`, each accompanied by a HuggingFace model card generated from `huggingface/MODEL_CARD_TEMPLATE.md` (intended use, training data, evaluation results, known limitations).
- All experiment logs on the public Weights & Biases project `anonymous/tinker-rl-bench`.

Checkpoints from JWT-interrupted Tinker runs are *not* uploaded, because those jobs did not reach a valid final state (Section 6.1).

14. **Crowdsourcing and Research with Human Subjects**

    (a) For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation? [NA] No crowdsourcing or human subjects research was conducted.

15. **Institutional Review Board (IRB) Approvals or Equivalent for Research with Human Subjects**

    (a) Does the paper describe potential risks incurred by study participants, whether compensation was provided to study participants, and whether informed consent was obtained? [NA] No human subjects research was conducted; IRB approval is not applicable.

16. **Declaration of LLM usage**

    (a) Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods? [Yes] LLMs appear in this work in two distinct roles, both declared:

    - **As subjects of evaluation.** The Qwen, Llama, Nemotron, GPT-OSS, and Kimi-K2 model families are the primary *subjects* of the benchmark; their role as RL fine-tuning targets is described in Sections 3 and 4.
    - **As authoring/editing aids.** GitHub Copilot was used for boilerplate experiment scripts and LaTeX/BibTeX scaffolding. ChatGPT Pro was used during revision to obtain reviewer-style critique of drafts; resulting suggestions were evaluated and, where accepted, manually incorporated. All scientific claims, experimental design, numerical results, figures, and statistical analyses are human-authored and independently verified against the logged Weights & Biases traces.

    LLMs are not an *original* or *non-standard* component of the core methodology.